中图分类号：TP311

论 文 编 号：10006ZY2206321

北京航空航天大學

硕 士 学 位 论 文

基于深度学习的软件漏洞检测关键技术研究

作 者 姓 名　王悦

学 科 专 业　计算机技术

指 导 教 师　高小鹏　教授

培 养 学 院　计算机学院

Empirical Research on Key Technologies for Software Vulnerability Detection Based on Deep Learning

A Dissertation Submitted for the Degree of Master

Candidate: Yue Wang

Supervisor: Prof.Xiaopeng Gao

School of Computer Science & Engineering

Beihang University, Beijing, China

中图分类号：TP311

论文编号：10006ZY2206321

硕　士　学　位　论　文

基于深度学习的软件漏洞检测关键技术研究

作者姓名　王悦　　申请学位级别　电子信息硕士

指导教师姓名　高小鹏　　职　　称　教授

学科专业　计算机技术　　研究方向　软件漏洞检测

一级学科　　学科方向

学习时间自　2022 年 09 月　01　日起至　2025 年　05　月　26　日止

论文提交日期　2025 年 05 月　16　日　论文答辩日期　2025 年　05　月　26　日

学位授予单位　北京航空航天大学　　学位授予日期　　年　　月　　日

关于学位论文的独创性声明

本人郑重声明：所呈交的论文是本人在指导教师指导下独立进行研究工作所取得的成果，论文中有关资料和数据是实事求是的。尽我所知，除文中已经加以标注和致谢外，本论文不包含其他人已经发表或撰写的研究成果，也不包含本人或他人为获得北京航空航天大学或其它教育机构的学位或学历证书而使用过的材料。与我一同工作的同志对研究所做的任何贡献均已在论文中作出了明确的说明。

若有不实之处，本人愿意承担相关法律责任。

学位论文作者签名：王悦　　日期：2025 年 05 月 30 日

学位论文使用授权书

本人完全同意北京航空航天大学有权使用本学位论文（包括但不限于其印刷版和电子版），使用方式包括但不限于：保留学位论文，按规定向国家有关部门（机构）送交学位论文，以学术交流为目的赠送和交换学位论文，允许学位论文被查阅、借阅和复印，将学位论文的全部或部分内容编入有关数据库进行检索，采用影印、缩印或其他复制手段保存学位论文。

保密学位论文在解密后的使用授权同上。

学位论文作者签名：王悦　　日期：2025 年 05 月 30 日

指导教师签名：高小鹏　　日期：2025 年 05 月 30 日

摘 要

随着软件系统复杂度的不断攀升，漏洞风险日益凸显，给隐私保护和系统安全带来严峻挑战。在此背景下，如何准确有效地检测源代码中的漏洞已成为软件领域的关键问题。当前主流的漏洞检测方法主要基于图神经网络或序列模型进行代码特征提取，并通过分类器完成漏洞判定。然而，这些方法在特征表征方面存在明显不足：现有基于图神经网络的方法难以建模远距离节点间的依赖关系，而序列模型则难以捕获代码的结构信息，导致特征学习不充分。为突破这些局限性，本文提出了两种创新性的解决方法。

方法一构建基于图神经网络的漏洞检测框架TreeGATTrans，通过多阶段处理实现高效特征学习。在特征预处理阶段，通过联合解析抽象语法树、控制流图和程序依赖图构建复合图结构，运用简化算法以降低图复杂度并保留关键漏洞路径。在特征学习阶段采用分层策略，在语句层面提取抽象语法树的语法特征，在跨语句层面利用图注意力网络捕捉结构依赖，最终通过注意力池化形成全局表征。该方法通过拓扑简化降低噪声干扰，利用注意力机制强化关键路径，并通过全局池化进一步捕获远距离依赖。

方法二构建图结构与预训练模型融合的漏洞检测框架GNNAdapter，设计了两种异构特征融合范式以增强代码表征的学习。图结构感知的浅层拼接融合（GCF）采用异构特征拼接的方式，实现代码结构表征与文本表征的静态融合；图结构引导的深度注意力融合（GAF）则以图结构为前缀，通过注意力机制完成结构表征与词元语义表征的动态交互。该融合机制有效整合代码的语法结构和语义上下文特征，弥补了单一表征方式的局限性。

实验结果表明，本研究提出的方法在多个基准数据集上均取得显著性能提升。基于图神经网络的漏洞检测框架TreeGATTrans在Devign、ReVeal与BigVul数据集上的F1值分别达到63.96%、42.25%和26.60%，显著优于传统基准模型。图结构与预训练模型融合的漏洞检测框架GNNAdapter在相同数据集上的F1值进一步提升至67.26%、54.87%和93.72%，全面优于现有最佳基线。实验结果表明，本研究所提方法通过有效学习漏洞相关的代码语法与语义特征，显著提升了漏洞检测的准确率和召回率。

关键词：漏洞检测，图神经网络，预训练模型，大语言模型

Abstract

With the continuous increase in software system complexity, vulnerability risks have become increasingly prominent, posing severe challenges to privacy protection and system security. Against this backdrop, how to detect vulnerabilities accurately and effectively in source code has become a critical issue in the field of software security. Current mainstream vulnerability detection methods primarily rely on graph neural networks or sequence-based models to extract code features, followed by classifiers to determine vulnerabilities. However, these methods exhibit significant limitations in feature representation: current graph neural networks struggle to model long-range node dependencies, while sequence-based models fail to effectively capture structural information in code, leading to insufficient feature learning. To overcome these limitations, this paper proposes two innovative solutions.

The first approach constructs a vulnerability detection framework called TreeGATTrans, which is based on graph neural networks and achieves efficient feature learning through multi-stage processing. During feature preprocessing, it builds a composite graph by jointly analyzing abstract syntax trees, control flow graphs, and program dependency graphs. Graph simplification algorithms are then applied to reduce complexity while preserving critical vulnerability paths. In the feature learning stage, a hierarchical strategy is adopted: at the statement level, syntactic features are extracted from abstract syntax trees, whereas at the inter-statement level, graph attention networks capture structural dependencies, ultimately forming a global representation through attention pooling. This approach reduces noise interference via topological simplification, enhances critical paths using attention mechanisms, and further captures long-range dependencies through global pooling.

The second approach constructs the vulnerability detection framework GNNAdapter by integrating graph structure with a pre-trained model, designing two heterogeneous feature fusion paradigms to achieve enhanced representation learning. The Graph-aware Concatenation Fusion (GCF) performs static fusion of code structure and textual representations via feature

concatenation, while the Graph-guided Attention Fusion (GAF) dynamically interacts structural features with token representations using attention mechanisms guided by graph structures. This fusion mechanism effectively combines syntactic structure and semantic contextual features, overcoming the limitations of single-representation approaches.

Experimental results demonstrate significant performance improvements across multiple benchmark datasets. The TreeGATTrans framework achieves F1-scores of 63.96%, 42.25%, and 26.60% on the Devign, ReVeal, and BigVul datasets, respectively, significantly outperforming traditional baselines. Meanwhile, the GNNAdapter framework further improves F1-scores to 67.26%, 54.87%, and 93.72% on the same datasets, surpassing all existing state-of-the-art models. These results confirm that the proposed methods effectively learn vulnerability-related syntactic and semantic features, substantially enhancing both detection accuracy and recall.

Keywords: Vulnerability Detection, Graph Neural Networks, Pre-trained Models, Large Language Models

目 录

第一章 绪论 1
1.1 研究背景和意义 1
1.2 国内外研究现状 2
1.2.1 传统的漏洞检测技术 2
1.2.2 基于神经网络的漏洞检测技术 3
1.3 研究目标及内容 8
1.3.1 基于图神经网络的漏洞检测框架 8
1.3.2 图结构与预训练模型融合的漏洞检测框架 8
1.4 本文组织结构 9
第二章 相关技术基础介绍 10
2.1 图神经网络 10
2.1.1 图神经网络简介 10
2.1.2 图神经网络主要类型 11
2.1.3 代码的图结构表示 14
2.2 预训练模型 17
2.2.1 预训练任务范式 17
2.2.2 代码预训练模型 19
2.3 图结构与预训练模型的融合 22
2.3.1 纯语言模型范式 23
2.3.2 图特征前缀注入 24
2.3.3 图结构与 LLM 联合建模 26
2.4 本章小节 26
第三章 基于图神经网络的漏洞检测框架 27
3.1 基于图神经网络的漏洞检测框架整体架构 27
3.2 代码属性图的自动化构建 27
3.3 基于树和图的层次化表征学习 31

3.3.1 基于树的代码语句表征学习 31
3.3.2 基于图的图表征学习 33
3.4 端到端训练与代码漏洞检测 36
3.4.1 交叉熵损失 36
3.4.2 焦点损失 37
3.4.3 三元组损失 37
3.5 本章小节 40
第四章 图结构与预训练模型融合的漏洞检测框架 41
4.1 图结构与预训练模型融合的漏洞检测框架整体架构 41
4.2 图结构与预训练模型融合的表征学习 43
4.2.1 图结构表征学习 43
4.2.2 图结构与代码文本的特征融合 43
4.3 端到端训练与代码漏洞检测 46
4.4 本章小节 47
第五章 实验结果与分析 48
5.1 基于图神经网络的漏洞检测 48
5.1.1 实验设置 48
5.1.2 数据集与基准模型 48
5.1.3 性能评价指标 50
5.1.4 实验环境 50
5.1.5 实验结果与讨论 51
5.2 图结构与预训练模型融合的漏洞检测 57
5.2.1 实验设置 57
5.2.2 数据集与基准模型 57
5.2.3 性能评价指标 58
5.2.4 实验环境 58
5.2.5 实验结果与讨论 59
5.3 本章小节 65

总结与展望 66

参考文献 68

致谢 73

插图清单

图 1　图卷积网络[61] 12
图 2　门控循环单元 13
图 3　图注意力网络[63] 14
图 4　源代码示例[34] 14
图 5　抽象语法树（Abstract syntax tree, AST）示例[34] 15
图 6　控制流图（Control flow graph，CFG）示例[34] 15
图 7　程序依赖图（Program dependence graph，PDG）示例[34] 16
图 8　代码属性图（Code property graph，CPG）示例[34] 17
图 9　CodeT5+模型示意图[65] 22
图 10　GraphLLM 模型结构示意图[51] 24
图 11　G-Retriever 模型结构示意图[68]（局部） 25
图 12　基于图神经网络的漏洞检测框架 28
图 13　简化前后的代码属性图 29
图 14　抽象语法树 33
图 15　基于图的表征学习 35
图 16　三元组训练数据集的构建过程 39
图 17　图结构与预训练模型融合的漏洞检测框架 42
图 18　图结构感知的浅层拼接融合 44
图 19　图结构引导的深度注意力融合 45

附表清单

表 1　漏洞检测数据集 54
表 2　实验环境配置 55
表 3　TreeGATTrans 与基线模型在各漏洞检测数据集上的性能对比 57
表 4　TreeGATTrans 消融树表征学习模块对漏洞检测性能的影响 58
表 5　不同图神经网络的性能对比 59
表 6　不同池化策略的性能对比 60
表 7　GNNAdapter 与基线模型在各漏洞检测数据集中的性能对比 64
表 8　GNNAdapter 消融图模块对漏洞检测性能的影响 66
表 9　预训练模型 CodeT5+与图结构的不同融合策略性能对比 67
表 10　不同预训练模型（codebert/phi-1）与图结构融合的漏洞检测性能对比 68

第一章 绪论

1.1 研究背景和意义

近年来，随着软件规模的持续扩大和功能的日益复杂化，软件漏洞的出现概率显著上升，给社会带来了严重的安全隐患和隐私风险。软件漏洞是指软件设计、实现或操作过程中的缺陷，可能被恶意利用进而损害系统功能、数据完整性或用户权益。以 2021 年 Microsoft Exchange Server 的 ProxyLogon 漏洞为例，攻击者通过该漏洞绕过身份验证窃取宏碁公司的机密数据，并提出了 5000 万美元的赎金要求[11]。同年 5 月，美国最大的燃油运输管道公司科洛尼尔遭受勒索软件攻击，其 5500 英里的输油管道系统被迫停运。此次网络攻击严重影响了美国东海岸的燃油供应，导致美国汽油价格飙升至七年来的最高水平，引发了民众恐慌。据 Cybersecurity Ventures 预测，全球网络犯罪成本将从 2015 年的 3 万亿美元激增至 2025 年的 10.5 万亿美元。此类漏洞攻击可能引发系统瘫痪、数据泄露、财产损失等连锁反应，甚至危及国家安全。

在开发早期准确识别潜在缺陷模块是提升软件可靠性的关键环节[11]。软件漏洞检测旨在通过分析软件属性构建检测模型，以判断模块是否存在漏洞或预测漏洞数量。这一研究方向对学术界和工业界的安全防御具有重要意义。

传统基于规则的漏洞检测方法主要依靠专家人工定义漏洞特征，其效果不仅受限于专家对漏洞规则的主观认知，而且极其耗费时间和人力。相较之下，基于机器学习与深度学习的检测技术能够自动学习和识别代码特征模式，在提升检测准确率与效率的同时，显著降低人工检测的工作强度与错误率。值得注意的是，早期研究（如 SySeVR[3]、Vuldeepecker[2]）通常将源代码片段视为自然语言序列，利用自然语言处理技术进行漏洞检测，但难以捕捉代码特有的结构信息。为了克服这一缺陷，研究者尝试从源代码中提取抽象语法树、控制流图或程序依赖图等代码的中间表示，然后通过图神经网络学习代码语义表征，例如 Devign[5]、ReVeal[6]等。然而这类方法存在节点表征同质化、远距离依赖捕捉不足等问题，且图神经网络固有的过平滑问题限制了其对深层语义的学习能力。尽管后续研究如 AMPLE[9]尝试通过图简化和核级表示模块解决此类问题，MVD+[40]尝试通过层次学习和图切片来捕获远距离节点间的依赖关系，但仍面临语法语义建模不充分和结构信息丢失等瓶颈。

自然语言处理（Natural Language Processing, NLP）技术的快速发展，尤其是以BERT[36]和 GPT-3[77]为代表的预训练语言模型，展现出强大的上下文建模与语义表征能力。这种能力不仅推动了自然语言处理任务的性能突破，也为代码漏洞检测提供了新的技术路径。研究者发现，代码与自然语言在语法结构、上下文依赖等方面存在相似性，这使得迁移预训练语言模型技术成为可能。LineVul[11]是其中的典型代表，通过预训练模型 CodeBERT[35]学习代码中的上下文和语法语义关系，显著提升了漏洞检测的性能。但需指出的是，当前方法虽在语义建模方面表现优异，但难以显式建模代码的结构语义，这一局限性限制了模型对代码整体逻辑的理解。

1.2　国内外研究现状

1.2.1　传统的漏洞检测技术

在软件工程领域，漏洞检测始终是核心研究课题之一。传统漏洞检测系统主要基于静态分析（Static Analysis）和动态分析（Dynamic Analysis）两类方法。两者在技术路径与使用场景上具有显著差异。

静态分析通过非执行方式对源代码或二进制文件进行自动化审查，重点检测代码结构中的潜在缺陷。该方法利用数据流分析、控制流分析与模式匹配等技术，识别缓冲区溢出、注入攻击及不安全数据处理等常见漏洞。此类工具能够高效扫描大规模代码库，从而扩大漏洞检测的范围[13]。在传统静态分析领域，研究者早期主要依赖专家预定义规则库与静态分析理论（如污点分析、抽象解释）进行漏洞检测。随着机器学习技术的发展，静态分析领域逐渐引入数据驱动的方法。2011 年，Shin 等[14]通过研究代码的静态复杂度量与执行复杂度量，并使用逻辑回归、J48 决策树、随机森林和朴素贝叶斯等机器学习方法构建漏洞检测模型。2013 年，Shar 等[15]提出了一种基于代码模式分析的漏洞检测方法，通过提取 SQL 注入与跨站脚本攻击的特征，并采用朴素贝叶斯、多层感知机和 C4.5 决策树算法构建检测模型，实验验证了其对两类漏洞的有效检测能力。2015 年，Perl 等[17]将代码度量与数据库中的元数据相结合，提出一种基于支持向量机的漏洞代码分析工具 VCCFinder，相比于传统分析工具 Flawfinder，其误报率降低了 99%。2016 年，Li 等[18]提出基于代码相似度算法的漏洞签名匹配机制，其工具 VulPecker 成功检测出 40 个未被 NVD 收录的零日漏洞（Zero-Day Vulnerabilities）。2018 年，Kronjee 等[19]

结合机器学习与污点分析技术，在 PHP 应用的 SQL 注入与 XSS 检测中实现精度突破。这些工具依赖人类专家设定的检测规则，并基于静态分析理论（如数据流、抽象解释和污点分析）进行程序分析。尽管已广泛应用，但存在局限性，一是过度依赖检测规则，无法发现规则未覆盖的漏洞；二是由于误报率较高，其准确性有待提升。

动态分析则通过监控程序运行时行为进行漏洞检测，重点关注输入验证与访问控制等动态执行逻辑。该方法通常在受控环境中运行目标程序，结合污点跟踪（Taint Tracking）技术追踪用户输入影响，或通过模糊测试（Fuzzing）生成异常输入触发程序崩溃以暴露漏洞。动态分析能够有效识别静态方法难以捕捉的运行时缺陷（如内存泄漏、竞态条件），但其检测效能受限于两大因素，一方面，动态路径覆盖的完备性依赖于测试用例生成质量；另一方面，实时监控带来的时间开销限制了大规模应用的可行性[76]。

1.2.2 基于神经网络的漏洞检测技术

在漏洞检测领域，传统机器学习方法主要依赖人工设计的软件指标作为核心特征。尽管这类方法被广泛采用，但其人工特征提取过程在效率与精度方面存在局限。近年来，研究者逐渐转向深度学习技术，通过自动学习历史数据中的漏洞模式，以提升检测性能。

深度学习模型通过端到端学习机制，能够直接从源代码等原始数据中自动提取语法语义特征，有效捕捉可能指示软件漏洞的复杂模式。这种能力在漏洞检测领域具有特殊价值，因为软件漏洞通常涉及非线性的代码特征和隐式的依赖关系。相较于传统方法，基于深度学习的漏洞检测展现出三重优势：（1）自动化特征学习能力，可自主分析大规模代码库与网络流量日志等结构化和非结构化数据，显著降低人工分析成本；（2）高效的计算并行性，支持对海量安全数据的实时处理；（3）卓越的泛化性能，能够识别训练数据中未显式包含的零日漏洞特征，通过表征学习实现未知漏洞的主动防御[13]。

当前基于深度学习的漏洞检测方法主要遵循两种技术路线：代码相似度匹配与语义模式挖掘。前者通过代码比对实现快速检测，但存在显著局限性：其一，代码相似性与漏洞存在性并非严格对应，导致高误报率；其二，对非克隆型漏洞（如由代码复杂性或特殊编程模式引发的漏洞）检测效能有限，且难以泛化至未知漏洞类型。鉴于此，本研究聚焦语义模式挖掘方法，其技术框架可大致分为基于序列、基于图结构以及基于预训练模型三类。需要说明的是，虽然预训练模型本质上属于序列建模范畴，但其通过海量数据训练获得的表征能力远超传统序列模型，因此本研究将其作为独立技术分支进行讨

论。后续章节将对这些方法的技术机理与研究进展进行系统性阐述。

1.2.2.1　基于序列模型的漏洞检测技术

自然语言与程序代码作为两种符号系统，在表达逻辑意图方面具有本质共性，二者都需要通过语法规则对基本元素进行合法组合以形成有效表达。这种内在相似性促使研究者将自然语言处理技术迁移至软件漏洞检测领域，当前多数研究均基于此技术路线展开。

Russell 等人[1]首先对 C/C++程序进行词法分析和规范化处理，利用 Word2Vec[20]生成代码元素的向量表征；随后分别通过卷积神经网络（Convolutional Neural Network，CNN）和循环神经网络（Recurrent Neural Network，RNN）捕获代码中的局部语义和远距离依赖；最终通过特征融合实现漏洞分类。该方法虽然能够识别部分漏洞特征，但由于未能显式建模程序的控制依赖和数据依赖关系，其表征复杂逻辑关联的能力存在局限。

为了提升漏洞定位精度与语义表征能力，Vuldeepecker[2]提出基于代码切片（code gadget）的检测方法。其首先识别函数调用等具有漏洞语法特征的代码元素，基于数据依赖关系生成语义关联的代码片段集合；随后通过 Word2Vec 生成代码元素的向量表征，并采用双向长短期记忆网络（Bidirectional Long Short-Term Memory Network, BiLSTM）实现特征提取与漏洞检测。该研究首次将代码切片粒度引入漏洞分析领域，但其控制流建模的缺失导致程序语义建模不充分。

SySeVR[3]在 Vuldeepecker 的基础上进行改进，通过增加数组操作、指针运算等高危代码元素的识别维度，同时考虑控制依赖和数据依赖生成更完备的代码切片集合，最后采用双向门控循环单元（Bidirectional Gated Recurrent Unit，BGRU[21]）网络进行特征学习。

当前基于序列模型的漏洞检测方法虽然放宽了对代码完整性和语法规范的要求，但其线性建模方式难以有效表征程序的控制依赖和数据依赖等结构化特征，在条件分支与循环结构等复杂逻辑的建模上存在固有缺陷。SySeVR 等基于代码切片的方法尝试提取相关依赖语句，但仍可能丢失关键上下文信息。此外，现有方法虽然能够捕捉局部语义特征，但难以建模深层语义，容易受到表面语法特征的干扰。

1.2.2.2　基于图模型的漏洞检测技术

编程语言与自然语言的核心差异在于其精确性与无歧义性，需通过编译器或解释器

生成可执行代码。现代主流编程语言均提供了成熟的编译器工具包，能够从源代码中提取抽象语法树、控制流图及程序依赖图等结构化语义信息[23]。基于这一特性，当前研究主要利用代码的图结构，通过图神经网络来学习程序的语法和语义特征。

Zhou 等人[5]提出复合代码表示框架 Devign，构建包含抽象语法树、控制流图、代码依赖图及自然代码序列（Natural Code Sequence, NCS）结构的联合图。其中抽象语法树节点为核心单元，不同边类型分别表示控制流、数据流与语法依赖关系。该框架利用门控图神经网络（Gated Graph Neural Network, GGNN[24]）聚合邻域节点信息，配合一维卷积层与全连接层实现图级分类，理论上可覆盖多样化漏洞模式。

与 Devign 类似，ReVeal[6]采用代码属性图作为输入，并利用 GGNN 学习图表征，最终由多层感知机进行二分类。针对真实场景中的样本不平衡问题，该研究引入合成少数类过采样技术 SMOTE[25]进行数据增强。此外，为了提升特征区分度，ReVeal 在损失函数中引入三元组损失，迫使漏洞样本与良性样本在向量空间中尽可能分离，从而增强模型的判别能力。

Li 等人[26]提出的 IVDetect 方法基于程序依赖图实现代码的表征，并采用特征注意力图卷积网络进行漏洞检测。为增强模型可解释性，该方法引入 GNNExplainer[27]工具，通过边掩码技术识别关键子图，从而直观呈现漏洞触发语句及其决策依据，为开发人员理解漏洞成因和修复代码提供有效支持。

为满足大规模代码漏洞检测需求，VulCNN[7]创新性地将函数级代码转换为图像表示。该方法首先对源代码进行归一化处理，并通过静态分析构建程序依赖图，其中每个节点对应函数中的一行代码。随后，采用 Sent2Vec[33]生成各代码行的向量表征，同时基于程序依赖图的中心性分析计算代码行的重要性系数。通过将重要性系数与代码向量相乘，获得代码行的最终表示。最终，将所有代码语句的向量表征矩阵作为图像输入卷积神经网络，以学习全局特征并进行漏洞检测。

Wang 等人[10]研究发现，现有模型学习到的特征普遍存在冗余且与漏洞关联性弱的问题，严重影响漏洞检测的准确性。为此，他们提出 DEEPVD 模型，通过强化类分离特征来提升模型对漏洞代码的区分能力。该模型创新性地融合了三个抽象层次的特征：语法层面的语句类型特征（类比自然语言的词性标注）、执行流程层面的后支配树（Post-Dominator Tree, PDT[29]）以及异常处理层面的异常流图（Exception Flow Graph, EFG[28]）。

在特征学习阶段，模型采用 GRU[30]处理代码序列，使用树形长短期记忆网络（Tree-LSTM[31]）分析抽象语法树，并利用带标签的图卷积网络（Label-GCN[32]）处理 PDT 和 EFG。最终通过特征拼接和多层感知机实现端到端的漏洞分类，显著提升了特征表征的判别性。

Wen 等人[9]的研究揭示了传统图神经网络在代码属性图建模中的两个关键局限：远距离依赖关系捕捉不足，以及异构边类型（如数据流边、控制流边）的语义差异利用不充分。虽然基于 GNN 的方法已取得显著进展，但其依赖局部邻域聚合的机制限制了全局信息表征能力，导致远程节点间的交互信息丢失。针对这些问题，AMPLE 框架提出了两个优化方案，一方面采用图简化技术，通过合并重复节点和冗余边来压缩代码属性图规模，降低节点间平均路径长度以改善远距离依赖建模；另一方面设计边感知图卷积网络（EA-GCN），在简化后的有向图中显式区分异构边类型，利用边类型权重矩阵动态调整消息传递过程中的邻域聚合权重，并引入多头注意力机制提升节点表征质量。为突破局部子图的信息限制，AMPLE 还设计了核级表示模块，通过并行使用大尺度卷积核（捕获跨模块远程依赖）和小尺度卷积核（提取局部结构特征），实现多粒度拓扑信息的融合与全局语义的学习。

Tang 等人[8]提出的 CSGVD 框架创新性地融合了序列与图结构信息，该框架首先利用 Joern[34]静态分析工具生成程序的控制流图，通过预训练模型 CodeBERT 初始化代码词元的向量表征，再采用 BiLSTM 来捕获序列上下文语义。同时，图卷积网络聚合邻域信息。最终采用平均双仿射注意力池化生成图表征完成分类。

尽管基于图神经网络的漏洞检测方法能够有效建模代码的结构特征，但其仍面临噪声干扰和远距离依赖捕获困难两大挑战。随着网络层数加深，GNN 的节点特征会因过度消息传递而趋于同质化，导致关键漏洞节点与普通节点的区分度下降，显著削弱模型的细粒度检测能力；同时，传统 GNN 的局部邻域聚合机制难以有效建模复杂依赖关系，特别是当程序存在嵌套循环或多分支结构时，关键的漏洞传播路径信息容易被稀释。

1.2.2.3　基于预训练模型的漏洞检测技术

预训练模型通过大规模代码语料库学习通用表征，有效缓解下游漏洞检测任务的数据稀缺问题。其核心优势在于迁移预训练阶段获得的泛化知识，从而增强模型对代码语义的深层理解能力。通过在下游任务进行微调，将预训练阶段习得的语法规则、逻辑模

式与数据依赖等知识迁移至漏洞检测任务，显著提升检测效率与泛化性能。

LineVul[11]提出了一种基于 CodeBERT 预训练语言模型的语句级漏洞定位框架。该框架首先利用基于 CodeSearchNet[37]语料库预训练的 CodeBERT 模型生成代码向量表征，通过 Transformer 模型[22]的自注意力机制捕获代码间的远距离依赖关系，并以[CLS]标记的最终层向量作为程序全局表征进行漏洞判定。同时，该模型通过分析注意力权重实现具体漏洞代码行的精确定位。

EPVD[12]提出路径级表征学习方法，其通过贪心算法将控制流图分解为从入口节点到出口节点的固定数目的关键路径，利用 CodeBERT 和 CNN 来学习路径表征，从而剔除程序中与漏洞无关的大量信息。具体而言，CodeBERT 捕获路径的内部注意力并学习其特征向量，CNN 则建模路径间的上下文关联，通过特征融合生成代码全局表征，最终由多层感知机完成漏洞分类。该方法通过路径筛选机制缓解预训练模型的输入长度限制，但可能因关键路径遗漏导致潜在漏洞特征丢失。

PDBERT[45]认为现有预训练目标对代码语义结构的建模存在不足，为此提出双任务预训练框架。控制依赖预测（CDP）任务要求模型基于代码片段预测语句级控制依赖关系，数据依赖预测（DDP）任务则聚焦词元级数据依赖关系建模。在预训练过程中，CDP 和 DDP 能够引导模型学习分析代码中细粒度依赖关系所需的知识。经过预训练后，模型在微调过程中能够更好地理解含有漏洞的代码，并能够直接对完整或部分函数进行依赖分析。

StagedVulBERT[69]构建基于多粒度语义建模的漏洞检测框架，通过融合代码局部与全局特征解决长序列代码的语义断层问题。其首先设计了分层表征模型 CodeBERT-HLS，使用 TETransformer 提取词元级语法特征，通过 Token2Statement 模块将词元向量聚合为语句级表征，再利用 SETransformer 来建模跨语句的上下文依赖关系，形成兼顾局部与全局的代码表征。其次，提出掩码语句预测预训练任务，通过随机遮蔽代码语句并重构其语义，增强模型对代码语法约束和跨语句数据流的理解能力。此外，引入滑动窗口分割与合并算法处理长代码序列，将输入代码分割为子片段后分别编码，再通过门控注意力机制融合子片段表征，缓解传统截断方法导致的信息丢失问题。该框架采用两阶段训练策略，先利用粗粒度函数级标签训练二元分类器筛选可疑函数，再基于细粒度语句级标签定位漏洞代码片段。

基于词元粒度进行预训练的序列模型，从大规模的代码语料库中学习到通用的代码特征与知识，在代码漏洞分类的下游任务上实现不错的性能效果。但其难以有效捕获程序的结构化信息，既无法显式建模控制依赖、数据依赖等关键特征，也缺乏追踪变量依赖和执行路径的能力，导致在处理需要复杂控制分析的漏洞类型时表现受限。虽然近期研究尝试引入结构预测的预训练任务，或通过不同粒度建模源代码以弥补这一不足，但如何实现真正的结构感知仍是亟待突破的核心难题。

1.3　研究目标及内容

本研究针对现有代码漏洞检测方法在结构语义建模中的核心挑战，提出两个创新框架以有效识别代码漏洞：基于图神经网络的漏洞检测框架，以及图结构与预训练模型融合的漏洞检测框架。此外，本研究以开源数据集 Devign[5]、ReVeal[6]和 BigVul[41]为基准，系统性评估框架的检测性能与泛化能力。

1.3.1　基于图神经网络的漏洞检测框架

针对传统方法在抑制细粒度语法噪声和建模远距离依赖关系方面的不足，本研究提出了一种基于树和图的层次化表征学习框架，具体包括：

（1）代码属性图的简化：研究如何通过节点归约与关键依赖保留策略，重构初始代码属性图的拓扑结构，在抑制细粒度噪声的同时保留控制依赖、数据依赖等关键语义路径。

（2）基于树和图的层次化表征学习：研究结合语句语法结构与跨语句依赖的分层表征方法，通过建立局部语法建模与全局语义传播的联合学习框架，有效解决传统图神经网络在复杂代码结构中难以捕获远距离依赖的问题。

（3）基于注意力机制的全局池化：研究基于注意力机制的全局池化方法，通过节点重要性加权聚合生成图表征，在保留漏洞关键节点信息的同时捕获远距离依赖关系。

1.3.2　图结构与预训练模型融合的漏洞检测框架

基于 Transformer 的预训练语言模型通过大规模代码语料训练，其能够捕捉各代码元素之间的语义关联，在代码摘要、补全等任务中展现出强大的语义理解能力。但序列化建模机制难以显式捕获控制依赖、数据依赖等结构化语义，而此类信息对跨语句漏洞模式的检测至关重要。图神经网络虽能通过拓扑聚合直接建模代码结构依赖，但其建模

复杂度有限。为解决以上局限性，本研究设计了两种图结构与预训练模型的融合策略：

（1）图结构感知的浅层拼接融合（GCF）：设计解耦式特征提取机制，分别通过预训练模型与图神经网络独立获取全局文本语义表征与图结构表征，并将其拼接实现浅层融合；

（2）图结构引导的深度注意力融合（GAF）：将图结构表征作为前缀提示，通过注意力机制实现代码结构特征与文本语义特征的深度融合。

1.4 本文组织结构

本章介绍了研究的背景与意义，系统回顾了国内外在漏洞检测技术方面的研究现状，包括传统技术及基于神经网络的方法，重点分析了序列模型、图模型和预训练模型的技术特性与局限性，并明确了本研究的目标与内容。

第二章详细阐述了本研究所涉及的主要技术，包括图神经网络的基本原理、类型及其在代码分析中的应用。同时介绍了预训练模型的架构、训练范式、在漏洞检测任务中的应用及其与图结构融合的前沿方法。

第三章提出了基于图神经网络的漏洞检测框架，系统性地阐述了从架构设计到具体实现的全流程。首先构建了整体技术框架，随后详细说明了代码属性图的自动化生成算法，并阐述了基于树和图的层次化表征学习方法，最终介绍了端到端训练机制与代码漏洞检测。

第四章构建了图结构与预训练模型融合的漏洞检测框架，重点探讨两者的协同表征学习机制，包括图表征学习模块的设计原理、图结构与代码文本特征的融合方法以及端到端训练机制与代码漏洞检测。

第五章开展了全面的实验验证与分析，分别从研究问题设定、基准模型选择、数据集构建、评价指标设计、实验环境配置、结果讨论与方法对比等维度进行展开。

总结与展望部分对全文进行了总结，并展望了未来的研究方向。

第二章　相关技术基础介绍

本章介绍本研究的技术背景，主要包括图神经网络、预训练模型以及图结构与预训练模型结合的相关技术与基础知识。

2.1　图神经网络

2.1.1　图神经网络简介

在计算机科学中，图（Graph）是一种重要的非线性数据结构，其数学表达形式为$G = (V, E)$，其中V为节点集合，$E \subseteq V \times V$ 为边集合。每条边$e \in E$关联一对节点 (u, v)，根据方向性可分为有向边和无向边。例如社交网络的单向关注关系属于有向边，而交通网络的双向通行是一种无向边。得益于这种灵活的表达能力，此数据结构已广泛应用于多个领域，例如社交网络分析（顶点表征用户实体，边映射人际关系）、知识图谱构建（顶点对应概念实体，边描述语义关系）、代码控制流分析（顶点表示语句，边指示执行路径）以及分子结构研究（顶点代表原子，边对应化学键）等。

随着大模型技术的演进，文本属性图（Text-Attributed Graph, TAG）因其融合图结构与文本语义的能力成为研究焦点。TAG 扩展定义为$G_s = (V, E, T)$，其中$T = \{t_v | v \in V\}$是节点文本特征的集合，通过将序列文本特征（如用户简介、商品描述和代码节点元素等）与图拓扑结合，为节点提供丰富的上下文语义信息，从而增强模型对复杂关系的理解能力。

图神经网络（Graph Neural Networks, GNNs）是一类专门处理图结构数据的深度学习模型，其核心思想是通过迭代聚合邻域信息来更新节点嵌入，以捕获图中的高阶依赖关系。具体而言，GNN 的第 l 层网络中，节点v的嵌入 $h_v \in R^d$ 更新公式为：

$$h_v^{(l+1)} = \psi\left(\phi\left(mh_{v'}^{(l)} : v' \in N(v)\right), mh_v^{(l)}\right) \tag{2.1}$$

其中 $N(v)$ 表示节点 v 的邻居集合，$\phi(\cdot)$ 和 $\psi(\cdot)$分别是聚合函数和更新函数。通过堆叠多层 GNN 网络，最终生成的节点嵌入可有效支持节点分类、图分类和链接预测等下游任务。

代码结构图建模是 GNN 的重要应用场景之一。由于代码天然具备图结构特征（如抽象语法树、控制流图、数据流图等），GNN 成为建模代码语义的理想工具。以代码属

性图（Code Property Graph, CPG）为例，其节点可表示变量、函数或语句，边则描述数据依赖和控制依赖等关系，此类结构可视为一种广义的文本属性图。通过多层信息传递，GNN 能同时捕捉代码的局部语法细节和全局执行逻辑，显著提升代码理解与漏洞检测的准确性。

2.1.2 图神经网络主要类型

图学习的早期方法以随机游走为核心，该方法通过模拟节点间的随机移动来捕获网络中节点的连通性和影响力分布。在此基础上，图嵌入技术进一步将节点或边表示为低维向量，其目标是通过路径采样保留图拓扑与节点关系。例如 DeepWalk[56]借鉴自然语言处理中的词向量思想，将随机游走路径视为“句子”进行训练；Node2Vec[57]则通过调节广度和深度优先搜索的偏置参数，平衡局部结构与全局拓扑结构的建模。这些方法为后续图分析任务（如社区发现、节点相似度计算）奠定了重要基础。

随着深度学习的发展，图神经网络在图学习领域取得显著进展，逐渐成为图学习的主流框架。GNN 通过端到端的信息传递机制，能够针对图结构特征动态优化模型。目前最具代表性的三类 GNN 变体包括：图卷积网络[61]（Graph Convolutional Networks, GCN）、门控图神经网络[62]（Gated Graph Neural Networks, GGNN）和图注意力网络[63]（Graph Attention Networks, GAT）。

图卷积网络作为图神经网络的基础架构，首次将卷积操作扩展至非欧几里得图数据空间，通过聚合节点的局部邻域信息实现高效特征提取。其核心思想是通过邻接矩阵A和节点特征矩阵对图进行卷积操作，生成新的节点表征（如图 1 所示）。具体的节点更新公式可表述为：

$$H^{(l+1)} = \sigma\left(\widetilde{D}^{-\frac{1}{2}}\tilde{A}\widetilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right) \tag{2.2}$$

其中$H^{(l)}$表示第l层的节点特征矩阵（输入层$H^{(0)} = X$），$W^{(l)}$为第l层可训练权重矩阵，σ是非线性激活函数，通常采用 ReLU。值得注意的是，邻接矩阵通过$\tilde{A} = A + I$添加自连接（I是单位矩阵），再基于度矩阵$\widetilde{D}$（对角元素为$\tilde{A}$的节点度数）执行归一化处理。该归一化操作$\widetilde{D}^{-\frac{1}{2}}\tilde{A}\widetilde{D}^{-\frac{1}{2}}$通过对邻接矩阵进行节点度数的平方根倒数缩放，有效控制特征传播中的梯度幅度。这种设计使得 GCN 既能通过加权平均捕获节点局部结构特征，又可通过多层堆叠实现跨阶邻域信息的层次化聚合，最终形成具有丰富语义的节点表征。

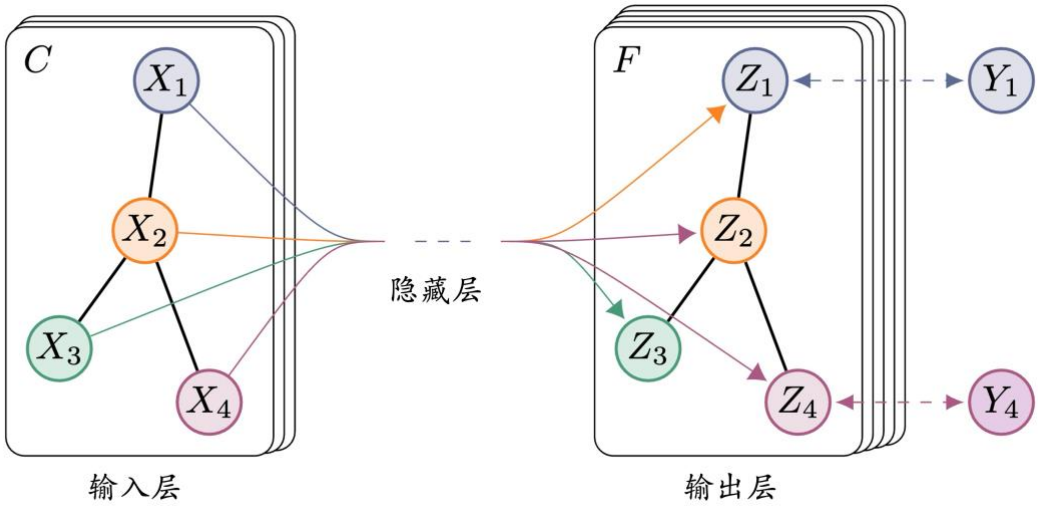


图 1　图卷积网络[61]

门控图神经网络是一种通过引入门控机制来增强信息传播能力的 GNN 变体。通过结合图的消息传播机制与序列建模中的门控策略，解决传统图网络在长期依赖建模中的梯度消失问题。其思想是将图中节点的状态更新过程视为一个循环时序操作，通过展开多时间步的消息传递，动态聚合邻居信息并自适应控制信息流。具体而言，GGNN 通过以下方式实现节点表征的更新：

$$h_i^{(t+1)} = \mathrm{GRU}\left(h_i^{(t)}, \sum_{j\in\mathcal{N}(i)}\left(W^{(t)}h_j^{(t)} + b^{(t)}\right)\right) \tag{2.3}$$

第$t+1$步时刻节点i的隐藏状态$h_i^{(t+1)}$由门控循环单元（GRU）计算得到，其输入为当前时刻的隐藏状态$h_i^{(t)}$以及邻居节点的加权聚合信息$\sum_{j\in\mathcal{N}(i)}\left(W^{(t)}h_j^{(t)} + b^{(t)}\right)$。其中$\mathcal{N}(i)$是节点$i$的邻居集合，$W^{(t)}$和$b^{(t)}$为可学习的权重和偏置参数。

门控循环单元（如图 2 所示）通过更新门$z_i^{(t)}$和重置门$r_i^{(t)}$动态调节信息流，更新门$z_i^{(t)}$控制历史状态的保留比例：

$$z_i^{(t)} = \sigma\left(W^z x_i^{(t)} + U^z h_i^{(t-1)} + b^z\right) \tag{2.4}$$

重置门$r_i^{(t)}$过滤历史状态中的冗余信息：

$$r_i^{(t)} = \sigma\left(W^r x_i^{(t)} + U^r h_i^{(t-1)} + b^r\right) \tag{2.5}$$

候选状态$\widetilde{h_i^{(t)}}$结合当前输入与重置后的历史状态：

$$\tilde{h}_i^{(t)} = \tanh\left(Wx_i^{(t)} + U\left(r_i^{(t)} \odot h_i^{(t-1)}\right) + b\right) \tag{2.6}$$

最终通过以下公式更新节点状态：

$$h_i^{(t)} = \left(1 - z_i^{(t)}\right) \odot h_i^{(t-1)} + z_i^{(t)} \odot \tilde{h}_i^{(t)} \tag{2.7}$$

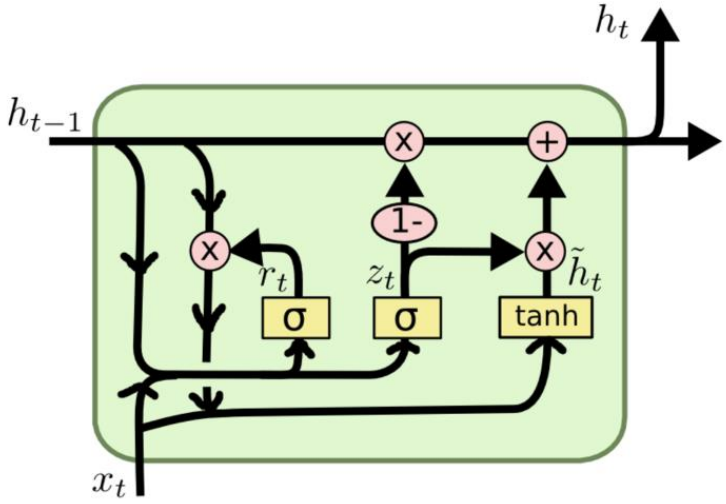


图 2　门控循环单元

这种门控机制使得 GGNN 能够有效平衡历史状态与当前信息，在多次迭代中稳定梯度传播，同时通过多步消息传递实现跨阶邻域信息的层次化聚合，从而形成具有丰富语义的节点表征。

图注意力网络通过将注意力机制引入图神经网络，有效解决了传统图卷积方法中固定权重聚合机制的局限性。其核心思想在于通过可学习的动态权重分配策略，使节点能够根据语义关联度自适应调整邻居节点的影响权重。GAT 的更新过程可以表示为：

$$h_i^{(l+1)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(l)} W^{(l)} h_j^{(l)}\right) \tag{2.8}$$

其中，$h_j^{(l)}$是节点j在第l层的特征表示，$W^{(l)}$是可学习的权重矩阵，$\alpha_{ij}^{(l)}$是节点i和其邻居j之间的注意力权重。具体而言，首先对节点特征进行线性投影，随后将投影后的特征向量经可学习参数向量 a 进行仿射变换，并通过 LeakyReLU 激活函数与 Softmax 归一化处理，最终生成具有语义感知的注意力权重（如图 3 所示）。具体可以形式化为：

$$\alpha_{ij}^{(l)} = \frac{exp(\mathrm{LeakyReLU}(a^T[W^{(l)}h_i^{(l)}||W^{(l)}h_j^{(l)}]))}{\sum_{k\in\mathcal{N}(i)} exp\ (\mathrm{LeakyReLU}(a^T[W^{(l)}h_i^{(l)}||W^{(l)}h_k^{(l)}]))} \tag{2.9}$$

其中 [· || ·]表示特征的拼接操作。为增强模型鲁棒性，GAT 进一步采用多头注意力机制并行学习多组独立的注意力权重，通过特征拼接或均值聚合实现多粒度语义融合。尽管注意力权重的动态计算增加了时间复杂度，但该方法能够显著提升模型对图结构中异质性关系的表征能力，在节点分类、图分类以及链接预测等任务中展现优越性能，特别是在处理度分布不均衡或局部结构稀疏的图数据时具有优势。

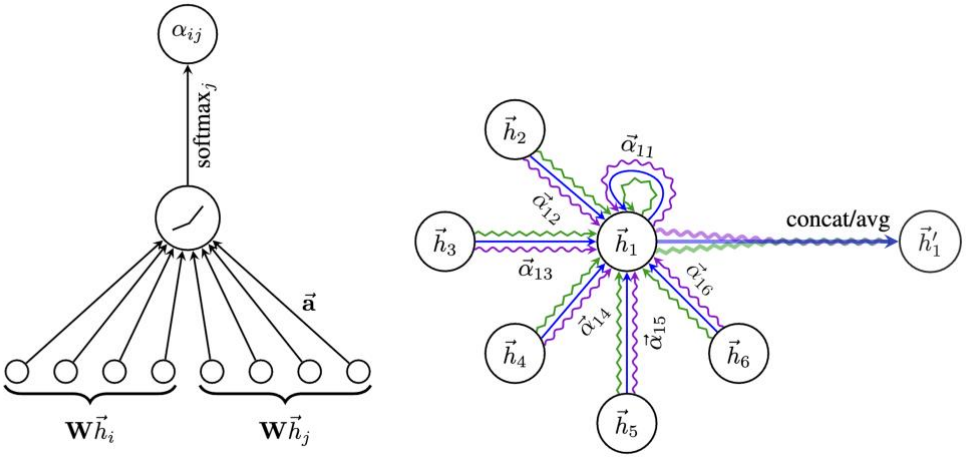


图 3　图注意力网络[63]

2.1.3　代码的图结构表示

近年来，图神经网络在代码分析和漏洞检测等领域得到了广泛应用。通过利用图的结构化表示，GNN 能够有效地捕捉程序中复杂的语法和语义关系。图的不同表示方式如抽象语法树、控制流图、程序依赖图和代码属性图提供了多层次的视角，使得 GNN 能够学习到更深层次的代码特征，从而提高漏洞检测的准确性。下文将详细介绍这几种代码图结构。

```
void foo()
{
    int x = source();
    if (x < MAX)
    {
        int y = 2 * x;
        sink(y);
    }
}
```

图 4　源代码示例[34]

抽象语法树（Abstract Syntax Tree, AST）通过树形结构表征代码的语法特征，其节点对应变量的声明、操作符的执行或语句的逻辑层级，边则体现代码元素的语法关系[42]。AST 的抽象特性能够清晰呈现程序逻辑结构，为 GNN 模型提供可解析的语法特征。如图 5 所示（其对应源代码图 4），AST 的层次化结构能够直观反映代码的语法组织特征，其树形拓扑通过父子节点关系显式地编码了程序元素的语法关联。这种结构化的表示方式使图神经网络模型能够有效地建模跨代码片段的语法模式，已在代码摘要生成、漏洞检测、代码理解[39]和程序分类等多个软件工程任务中展现出显著优势。

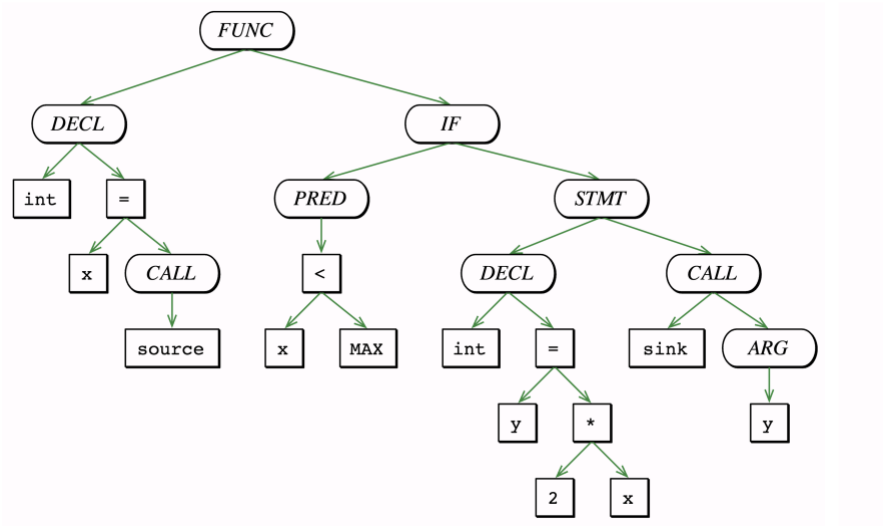


图 5　抽象语法树（Abstract syntax tree, AST）示例[34]

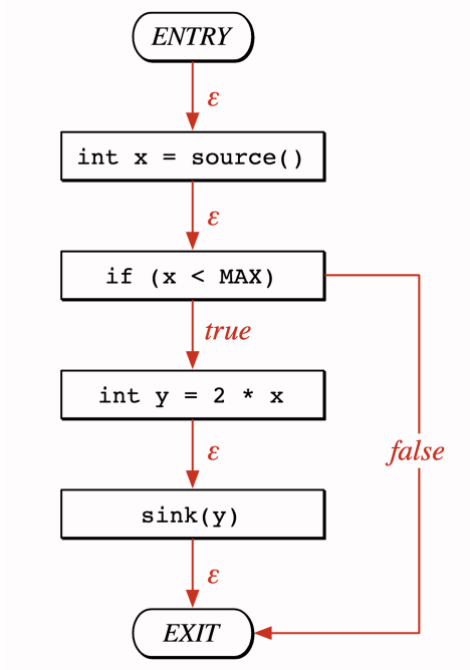


图 6　控制流图（Control flow graph，CFG）示例[34]

控制流图（Control Flow Graph, CFG）则为程序的执行路径提供了详细的图结构表示。CFG 的每个节点表示基本的代码语句，边则表示控制流的转移。通过分析控制流，GNN 能够识别程序中条件语句、循环和跳转等控制逻辑，这对于漏洞检测至关重要，因为许多漏洞与程序的控制流逻辑密切相关（如死锁和无限循环等）。源代码对应的 CFG 如图 6 所示，直观地展示了程序的控制流路径。

程序依赖图（Program Dependence Graph, PDG）通过整合数据依赖和控制依赖关系，构建了程序语义关联的完整表示[43]。数据依赖关系刻画了变量间的值传递过程，例如某个变量的计算结果被后续操作所使用；而控制依赖关系则反映了程序执行流的条件约束特性，如特定代码段的执行受限于前驱节点的分支判断。如图 7 所示，PDG 的边集同时包含了体现数据流动的定义-使用链和控制约束的条件跳转路径。基于这种双重依赖关系的建模，图神经网络能够有效捕捉潜在的漏洞触发模式，包括未经验证的外部输入通过数据依赖关系传播的路径，以及关键安全操作因控制依赖关系缺失而可能被绕过的风险场景。这种融合数据流和控制流的综合分析框架，提升了针对输入验证类漏洞（如 SQL 注入、缓冲区溢出等）的检测能力。

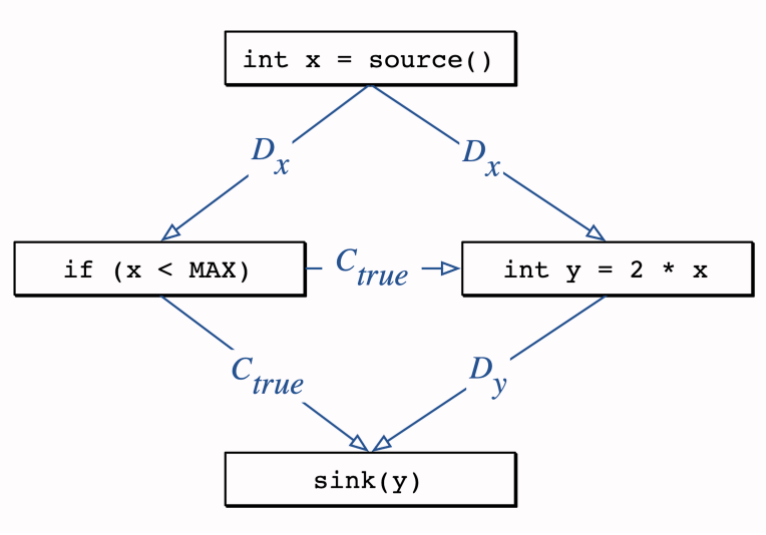


图 7　程序依赖图（Program dependence graph，PDG）示例[34]

代码属性图（Code Property Graph, CPG）是将 AST、CFG 和 PDG 融合而成的一种综合性图表示[44]，其包含程序的语法、控制流和数据流等信息。多维信息整合使得 GNN 更全面地理解程序的结构与行为，能够学习到更复杂的漏洞模式，从而在实际应用中表现出更高的准确性。源代码对应的 CPG 如图 8 所示，综合展示了程序的多种信息。

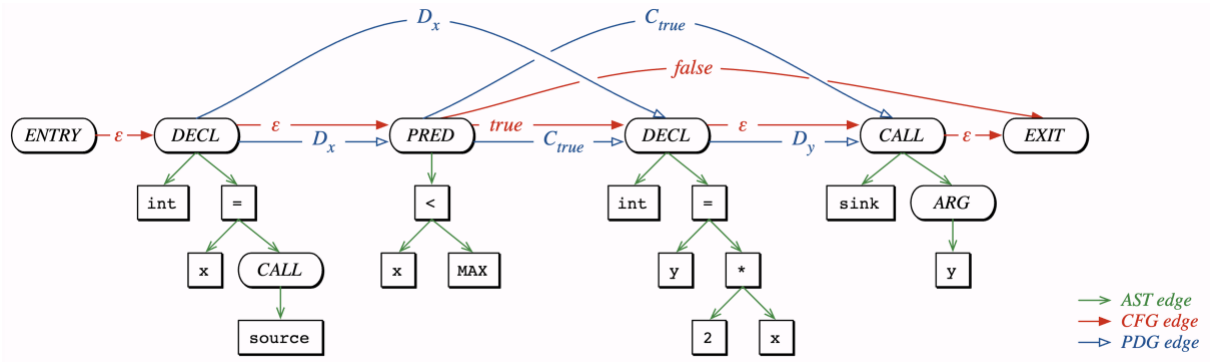


图 8　代码属性图（Code property graph，CPG）示例[34]

AST 虽能反映代码的语法结构，但缺乏语义层面的上下文信息；CFG 虽可直观呈现程序执行路径，却缺失关键的数据流特征。相比之下，PDG 和 CPG 整合了控制依赖与数据依赖关系，能够更全面地反映代码的语法和语义特征，其中 CPG 具有更丰富的代码表征维度。因此，本研究采用图神经网络对 CPG 进行建模分析。考虑到 CPG 结构的复杂性，需设计专门的预处理流程，以有效提取与漏洞检测相关的关键特征。

2.2　预训练模型

2.2.1　预训练任务范式

预训练语言模型作为自然语言处理的核心技术，其本质是通过统计建模估计语句中词元的概率分布。随着模型参数规模突破百亿量级，大型语言模型（Large Language Model, LLM）在机器翻译、文本摘要和文本生成等任务中展现出卓越性能。现代 LLM 均基于 Transformer 架构构建，该架构通过查询（Query）、键（Key）和值（Value）注意力机制实现词元序列的信息聚合，其数学表达式为：

$$\text{Atettention}(Q,K,V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})\text{V} \tag{2.10}$$

其中，Q，K，$V \in \mathbb{R}^{n\times d_k}$分别表示查询、键和值的矩阵，$d_k$为键向量的维度。通过在超大规模语料库上进行预训练，LLM 能够学习到丰富的语义表征，这为下游任务提供了高效的参数初始化基础。根据注意力机制的方向性，主流 LLM 可分为因果语言建模（Causal Language Modeling, CLM）与掩码语言建模（Masked Language Modeling, MLM）两种范式。

2.2.1.1　因果语言建模

自回归模型（如 GPT-3[77]、LLaMA[78]等）通过因果语言建模方法，基于前序词元预测当前位置的概率分布。具体而言，给定输入序列 $x = [x_1, x_2, ..., x_n]$，模型通过链式法则将联合概率分解为条件概率的乘积，目标函数为最大化序列条件概率：

$$\mathrm{P}(x) = \prod_{i=1}^{n} P_\theta(x_i|x_{<i}) \tag{2.11}$$

对应的负对数似然损失函数为：

$$\mathrm{L} = -\sum_{i=1}^{n} \log P_\theta(x_i|x_{<i}) \tag{2.12}$$

其中，$x_{<i}$表示在位置i之前的所有词元，θ为可训练模型参数。为实现这种单向依赖关系，模型在 Transformer 架构的注意力计算中引入下三角掩码矩阵，以确保x_i只能关注之前的词元。值得注意的是，训练阶段通过并行计算所有位置的交叉熵损失实现高效优化，而推理阶段则采用自回归方式逐词生成序列。

这种序列生成机制赋予模型构建远距离语法结构的能力，在文本生成任务中展现出显著优势。通过调节温度参数（控制输出分布的尖锐程度）与采样策略（如 top-k 抽样或核采样），可有效平衡生成文本的创造性与可控性。这种灵活性使得模型能够适应不同场景需求，例如在创意写作中提高温度值以增强多样性，或在技术文档生成中采用贪婪解码确保准确性。但受限于单向上下文建模，CLM 在处理全局依赖任务时存在信息缺失风险，且过度依赖前序内容可能导致生成文本的语义一致性不足。

2.2.1.2　掩码语言建模

掩码语言模型采用双向上下文来增强对文本理解能力。以 BERT 为代表的模型在预训练阶段采用动态掩码策略，具体操作是随机选择 15%的输入词元进行掩码处理，其中 80%替换为特殊词元[MASK]，10%替换为随机词元，剩余 10%保持原词不变。模型通过最大化被掩码词元的条件概率完成训练，其目标函数表示为：

$$L = -\sum_{m \in M} log\ P(m|\hat{x}) \tag{2.13}$$

其中M为被掩码词元的集合，$\hat{x}$表示含有掩码标记的输入序列。例如，当输入序列为$\hat{x} = [[\mathrm{CLS}], x_1, [\mathrm{MASK}], x_3, [\mathrm{MASK}], x_5, [\mathrm{EOS}]]$，模型需要同时利用左右双向上下文预测被掩码位置的内容。

这种设计通过强制模型建立跨位置语义关联，既提升了上下文建模能力，又缓解了预训练与微调阶段的表征差异问题。得益于双向注意力机制，MLM 在文本分类、问答系统等需要全局语义理解的任务中展现出卓越性能。然而，传统 MLM 仅针对少量掩码词元（约 15%）进行训练，存在训练效率低下的问题。为优化训练效率，Clark 等人[64]提出 ELECTRA 模型，其通过判别输入中的每个词元是否被模型替换来构建训练目标，从而实现对全部输入词元的损失计算。

2.2.1.3 去噪语言建模

以 GPT 为代表的因果语言模型虽擅长自回归生成，但因缺乏双向上下文建模能力，难以胜任翻译、摘要等序列到序列的任务。而 BERT 类型的掩码语言建模通过双向编码器获取全局表征，却因预训练任务与生成目标的失配，限制了其在生成任务中的直接应用。为解决这一问题，经典编码器-解码器架构通过联合优化双向编码与自回归解码，实现了生成与理解的统一。

T5 模型[79]首次系统化提出跨度破坏（Span Corruption）预训练目标：将输入文本中的连续片段替换为唯一哨兵标记（如[extra_id_i]），构造含噪输入$\hat{x}$，并令解码器自回归生成由哨兵标记引导的原始片段序列$\{[\text{extra_id_1}], span_1, \dots, [\text{extra_id_k}], span_k\}$，其训练目标可形式化为最大化条件概率：

$$\max_{\theta} p_\theta(y|\hat{x}) = \prod_{i=1}^{n} p_\theta(y_i|\hat{x}, y_{1:i-1}) \tag{2.14}$$

其中θ为模型参数，$\hat{x}$为含噪声的输入，$y_{1:i-1}$为已生成前缀。这一机制本质上是 MLM 的泛化形式，即当掩码跨度为单个词元时，T5 退化为标准 MLM 任务；而当整个输入被掩码为单一跨度时，则等价于 CLM 任务。

2.2.2 代码预训练模型

代码预训练模型作为软件工程与自然语言处理的交叉研究方向，通过海量代码库的自监督学习获取通用代码表征能力，在代码理解与生成等下游任务中展现出显著性能优势。程序语言在结构化程度、语义明确性和逻辑严谨性等方面与自然语言存在显著差异。虽然其词汇系统基于英语构建，但必须严格遵循形式化的语法规则才能确保正确编译执行。特别地，高质量代码通常与其自然语言注释和文档保持语义一致性，这种跨模态的

关联性为模型学习提供了丰富的对齐监督信号。基于这些特性，研究者不仅借鉴了自然语言处理中的掩码语言建模等技术，还专门设计了针对代码结构特性的预训练目标。根据模型架构差异，现有代码预训练模型主要分为编码器（Encoder）、解码器（Decoder）以及编码器-解码器（Encoder-Decoder），下文将系统分析各类方法的技术原理与典型实现方案。

2.2.2.1　编码器

在自然语言理解领域取得突破性进展的 Transformer 编码器（如 BERT）被成功迁移至代码处理领域后，研究者开始探索面向程序语言特性的方法。例如，CodeBERT 采用双目标训练范式，在 CodeSearchNet 等代码与自然语言的混合语料上，通过掩码语言建模捕捉语义特征，同时利用替换标记检测（Replacement Token Detection, RTD）提升抗噪能力。该模型创新性地构建代码片段与对应文本描述的输入对，使模型能够联合学习代码语义及其与自然语言的关联。目前，基于 CodeBERT 改进的多个方法已成功应用于漏洞检测任务（例如 LineVul、CSGVD 和 PILOT 等），展现出优异的代码表征能力。为验证所提方法的泛化性，本研究选择 CodeBERT 作为编码器的代表性模型，在 5.2.5 节将展示其与图结构融合的实验结果。

针对代码的结构特性，研究者设计了辅助任务以增强模型理解能力。在 MLM 的基础上，GraphCodeBERT[71]新增两项结构感知任务，其通过变量依赖图预测逻辑边关系，以及对齐代码文本与结构表征，并采用数据流图的邻接矩阵作为注意力掩码约束无效变量交互。SynCoBERT[72]则通过标识符预测建模符号上下文关联，利用抽象语法树边预测捕捉语法节点拓扑依赖，同时引入三模态对比学习框架，使代码、注释与 AST 在隐空间形成语义等价表征。PDBERT 进一步提出细粒度依赖预测任务，分别从语句级控制流和标记级数据流两个维度解析程序依赖关系。这类辅助任务通过显式注入语法规则、控制依赖等先验知识，显著提升预训练模型的表征在下游任务中的迁移能力。

2.2.2.2　解码器

随着 GPT-3 通过上下文学习能力验证了仅解码器架构的 Transformer 模型在语言建模中的有效性，该架构迅速成为自然语言处理领域的主流范式。这种技术趋势也延伸至代码处理领域，催生出包括 CodeGen[80]、Phi-1[70]以及 Code Llama[73]等多个基于因果语言建模的预训练模型。

Meta 研发的 Code Llama 系列模型则代表大规模参数路线的典型成果。该系列基于 Llama 2 架构，首先进行通用代码预训练，然后进行 Python 专项微调，最后强化指令遵循能力。模型提供从 7B 到 70B 不同参数规模的多个版本，不仅在多语言代码生成基准测试中持续保持领先优势，同时支持 10 万词元级上下文窗口的代码填充功能，能够高效处理复杂代码补全任务。这一系列创新使 Code Llama 成为当前代码生成领域最具影响力的模型之一。

在参数效率优化方向，Gunasekar 等学者开发的 Phi-1 模型取得了显著突破。该模型仅采用 13 亿参数规模，训练数据总量控制在 70 亿词元，其中 60 亿取自 StackOverflow 技术社区，剩余 10 亿由 GPT-3.5 合成生成。通过实施创新的数据质量提升方案，包括严格筛选代码片段、生成合成教科书数据以及构建专项练习数据集，该模型在主流代码生成基准测试中展现出与参数量级大数十倍的竞品相当的性能表现，成功建立了小规模模型的高效训练范式。基于 Phi-1 在参数效率与性能表现上的突出优势，本研究将其作为预训练模型基座，在 5.2.5 节通过图结构融合实验进一步验证所提方法的泛化能力。

2.2.2.3 编码器-解码器

编码器-解码器型大语言模型通过整合编码器的语义解析能力与解码器的序列生成能力，实现高效的语言处理。编码器通过注意力机制将输入文本映射至隐空间表征，深度解析语法结构与语义依赖关系，生成具有泛化能力的紧凑语义向量。这种隐含表示作为中介语言，不仅弥合了不同输入和输出格式之间的差异，还为解码器提供了细粒度的上下文信号。解码器则基于隐空间表征展开自回归预测，输出符合目标领域规范的文本序列。

编码器-解码器架构凭借其卓越的双向语义理解与生成能力，已成为软件工程领域众多先进代码模型的核心基础，包括 CodeT5+[65]和 AlphaCode[81]等代表性工作。Salesforce AI Research 团队开发的 CodeT5+作为新型开源代码大语言模型，通过创新的架构设计和多任务预训练策略显著提升了代码理解与生成性能。如图 9 所示，该模型采用灵活的编码器-解码器架构，支持模块化组件组合以适应多样化代码任务。其预训练过程融合了跨度去噪、对比学习、文本代码匹配和因果语言建模等目标，覆盖单模态和多模态代码数据，有效缩小了预训练与下游任务之间的差距。

具体来说，CodeT5+采用分阶段预训练策略，首先在大规模单模态代码数据上进行

训练，随后在较小规模的双模态代码文本数据上继续优化。为提升训练效率，模型采用冻结预训练参数的方法，例如使用 CodeGen-mono 模型初始化解码器，较小规模的 CodeGen 模型初始化编码器，大幅减少了可训练参数量。通过引入指令微调技术，模型在合成指令数据上训练后展现出优异的自然语言指令理解能力，在零样本场景下表现突出。实验结果表明，CodeT5+在 20 余个代码相关基准测试中全面领先，涵盖代码生成、补全、检索和总结等任务。在数学编程和代码总结任务中，其小规模模型也显著优于许多大规模语言模型。基于这些优势，本研究选用 CodeT5+的编码器部分与图结构进行融合，构建高效的漏洞检测框架。

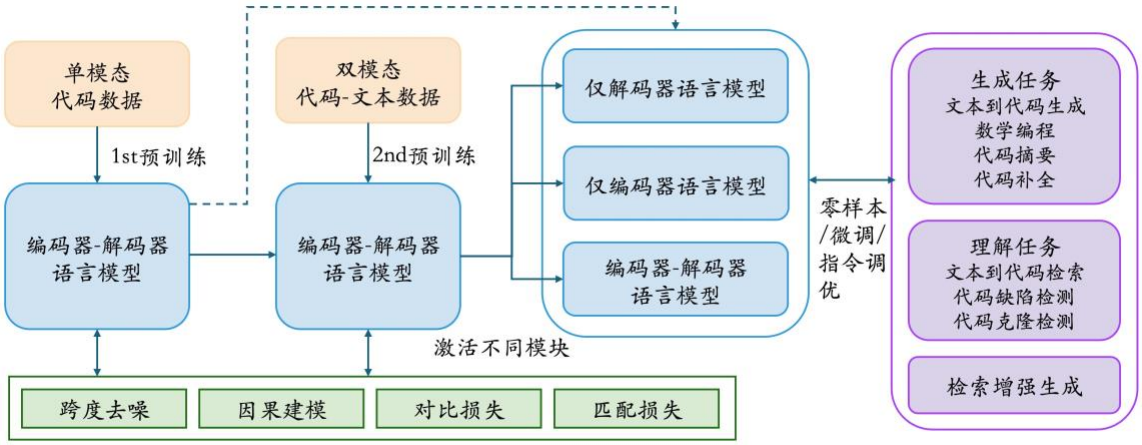


图 9　CodeT5+模型示意图[65]

2.3　图结构与预训练模型的融合

大语言模型推动了人工智能技术的革新，在自然语言处理领域取得突破性进展。这类模型通过大规模模型架构和海量数据训练，已成功应用于计算机视觉、推荐系统等多个领域。以 1750 亿参数的 GPT-3 为例，该模型不仅能生成文本，还能完成复杂问题求解和代码生成等任务。基于万亿级词元的预训练，大语言模型掌握了跨领域的知识表征和推理能力，在语言理解和知识密集型任务中表现突出。其特有的上下文学习机制使模型在少样本（Few-shot）甚至零样本（Zero-shot）条件下仍能有效适应新任务，为下游任务提供了高效的迁移方案。

大语言模型虽具备通用智能潜力，但仍受限于事实准确性不足、现实约束缺失和可解释性差等问题。研究表明，图结构能有效组织高质量事实知识，其与语言模型的结合

可显著提升推理能力并减少幻觉现象[50]。当前研究主要采用三种图数据处理方法：（1）纯语言模型驱动范式（LLM-Only）：将图结构的邻域和拓扑信息转换为自然语言描述后直接输入语言模型；（2）图特征前缀注入（GNN as Prefix）：使用图神经网络编码图结构特征，并将生成的表征序列作为语言模型的输入前缀；（3）图结构与 LLM 联合建模（LLM-GNN Integration）：通过联合训练机制，实现图神经网络结构编码器与语言模型语义解码器的深度整合[58]。

2.3.1 纯语言模型范式

纯语言模型范式致力于引导大语言模型直接解析自然语言描述的图结构信息，并基于此类信息完成下游推理任务。该方法无需对模型参数进行微调，其核心在于设计符合语言模型认知模式的文本提示，将图节点、边及其拓扑关系转化为序列化描述，从而激发预训练 LLM 的隐式结构理解能力。

由于图数据的离散拓扑特性与文本的连续序列特性存在本质差异，此类方法需突破两个关键技术难题：其一，如何有效实现图结构到自然语言的转换；其二，如何确保语言模型能够准确解码文本化图表征的语义信息。当前研究主要通过提示工程（Prompt Engineering）技术，在文本空间内构建结构化描述模板，以探索预训练 LLM 的结构推理潜能。提示工程通过设计包含图邻接关系、节点属性等要素的文本提示，引导模型关注结构化语义模式。例如，Hu 等人[46]通过设计不同的提示方式，探索了大型语言模型在结合图结构信息进行节点分类、边预测和图分类任务时的能力。具体提出了两种利用结构信息的方法，第一种直接将所有邻居节点的数据输入到 LLM 中，第二种使用基于检索的提示来引导 LLM 仅关注相关的邻居数据。Wang 等人[47]提出了两种基于指令的提示技术，以增强大语言模型的图形推理能力。首先，通过在图形文本描述后添加“让我们先构建一个带有节点和边的图形”来帮助模型可视化图形结构，从而更好地理解问题。其次，在上下文示例之前提供相关算法的细节，例如针对最短路径任务使用深度优先搜索算法，以帮助模型理清解决方案的思路。这两种方法可以与上下文学习和链式思维提示结合使用，共同提高 LLM 在图形推理任务中的表现。

进一步地，Guo 等人[48]利用图描述语言（Graph Description Language, GDL）将图结构化数据转化为可读的文本描述，然后利用图提示处理程序将用户查询与 GDL 结合，生成 LLM 的输入，从而实现推理和答案生成。在漏洞检测场景，Liu 等人[67]通过将代码

的数据流图（Data Flow Graph, DFG）作为代码特定信息整合到提示中，来增强 LLM 在软件漏洞检测中的性能。具体而言，首先使用 Tree-sitter 库提取代码的抽象语法树，然后基于抽象语法树检测变量及其依赖关系以构建 DFG，最后将生成的 DFG 整合到提示中，帮助 LLM 更好地理解代码的结构和数据依赖关系，从而提高漏洞检测的准确性。GRACE[16] 则采用图结构信息和上下文学习相结合的方式提升基于大型语言模型的漏洞检测效果。该系统包含三个核心模块。演示选择模块基于语义、语法和词汇相似性筛选最相关的代码示例，确保模型能够学习到与目标代码高度相关的上下文信息。图结构信息生成模块借助代码属性图融合控制流、数据流和语法结构信息，使模型能够更全面地理解代码的结构和语义。增强漏洞检测模块则通过定制化提示设计引导大型语言模型实现更精准的漏洞检测，进一步提升了模型在漏洞检测任务中的性能和准确性。

2.3.2　图特征前缀注入

尽管通过自然语言的隐式结构描述方法已取得一定成效，但其仍面临两个核心挑战。首先，大语言模型的输入长度限制导致难以捕获全局结构信息，且过长的上下文可能削弱其推理性能。其次，针对不同任务和数据集需设计特定提示模板，而人工构建的提示模板往往缺乏跨数据集的泛化能力。为此，研究者开始探索将图神经网络作为结构编码器，通过对齐图神经网络生成的图表征与自然语言语义来增强模型理解[49]。具体而言，GNN 充当图数据的分词器，将节点特征编码为携带结构信息的图表征序列，进而输入大语言模型进行语义融合。对于图分类等需要全局信息的任务，需通过池化操作将节点嵌入聚合为图级表征，并采用前缀调优（Prefix Tuning）或提示调优（Prompt Tuning）策略将其融入语言模型。

前缀调优通过向 Transformer 各层的注意力键值空间中注入可训练的前缀参数，逐层调节模型的注意力分布，以实现对模型推理路径的层级化调控。具体而言，给定包含 L 层 Transformer 结构的预训练大语言模型，该方法在每层注意力模块的键和值张量前动态拼接 K 个可优化前缀向量。以第 l 层注意力机制为例，定义该层前缀张量为$P_l \in \mathbb{R}^{K\times d_M}$，其中$d_M$为模型隐藏层维度。将前缀张量与原始键矩阵$K_l \in \mathbb{R}^{N\times d_M}$以及值矩阵$V_l \in \mathbb{R}^{N\times d_M}$进行纵向拼接，以实现参数注入：

$$K_l' = [P_l; K_l] \in \mathbb{R}^{(K+N)\times d_M}, V_l' = [P_l; V_l] \in \mathbb{R}^{(K+N)\times d_M} \tag{2.15}$$

经前缀扩展的键值对将参与该层的注意力计算，修正后的计算过程可形式化为：

$$O_l = \mathrm{Attn}(Q_l, [P_l; K_l], [P_l; V_l]) \tag{2.16}$$

全局前缀参数通过可训练的张量$P \in \mathbb{R}^{L\times K\times d_M}$进行初始化，其中 L 表示模型总层数，K 为每层前缀长度。例如，Chai 等人[51]提出的 GraphLLM 框架通过引入图增强前缀提示来提升大语言模型在图推理任务中的能力。如图 10 所示，该方法通过对图表示进行线性投影生成图增强前缀，并将其添加到 LLM 的每个注意力层中，以提供额外的上下文信息。这种图增强前缀使得 LLM 能够在推理过程中更有效地理解图的结构和节点之间的关系，从而生成更准确的自然语言响应。

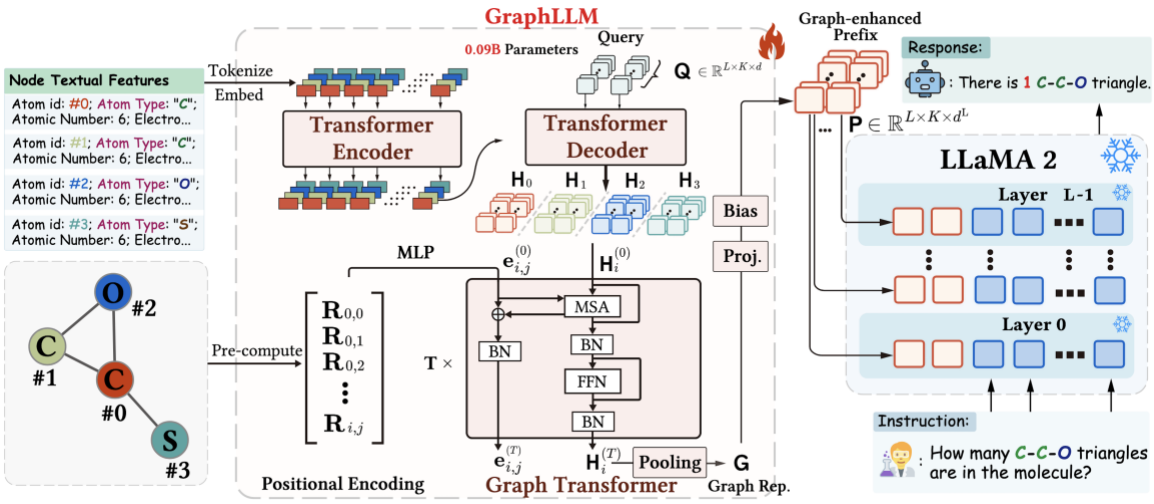


图 10　GraphLLM 模型结构示意图[51]

提示调优方法则采用不同的任务适配策略，其核心思想是通过在输入嵌入层首部添加连续型提示向量，引导模型理解任务语义[52]。给定输入标记序列$X = \{x_1, \dots, x_n\}$与提示序列$P = \{p_1, \dots, p_q\}$，模型输出序列$Y = \{y_1, \dots, y_r\}$的条件概率分解为：

$$p_\theta(Y|X, P) = \prod_{i=1}^{r} p_\theta(y_i|y_{<i}, [P; X]) \tag{2.17}$$

其中拼接后的输入表征$[P; X] \in \mathbb{R}^{(q+n)\times d_l}$直接影响模型对任务语义的解析。其中，软提示调优通过可学习参数取代人工设计离散提示，$y_{<i}$表示序列y在位置 i 之前的前缀，$p_\theta(y_i|y_{<i}, [P; X])$表示在给定$y_{<i}$和$[P; X]$条件下生成词元$y_i$的概率。与层级化操作的前缀调优不同，提示调优仅需优化低维提示矩阵$P_e \in \mathbb{R}^{q\times d_l}$，通过修改输入表征的初始分布实现任务适配。典型应用如 He 等人[68]提出了一种检索增强生成（Retrieval-Augmented Generation, RAG）框架 G-Retriever，通过将图检索与生成任务相结合，将图结构融入大

语言模型。如图 11 所示，其从大规模图中检索出与问题最相关的子图，并通过图神经网络编码子图生成结构化表示，再以软提示的方式输入到 LLM 中。该方法有效减少了幻觉现象，提升了模型在大规模图上的扩展性和可解释性，同时显著提高了文本图问答任务的准确性和效率。类似地，本研究考虑采用提示调优方法，将图结构编码为向量提示输入预训练模型。具体实现中，保持输入层提示符可训练以更新图神经网络模型参数，同时微调模型参数。该方法在保持模型原有语义理解能力的同时，使图结构与模型参数更好对齐，从而提升代码漏洞检测性能。

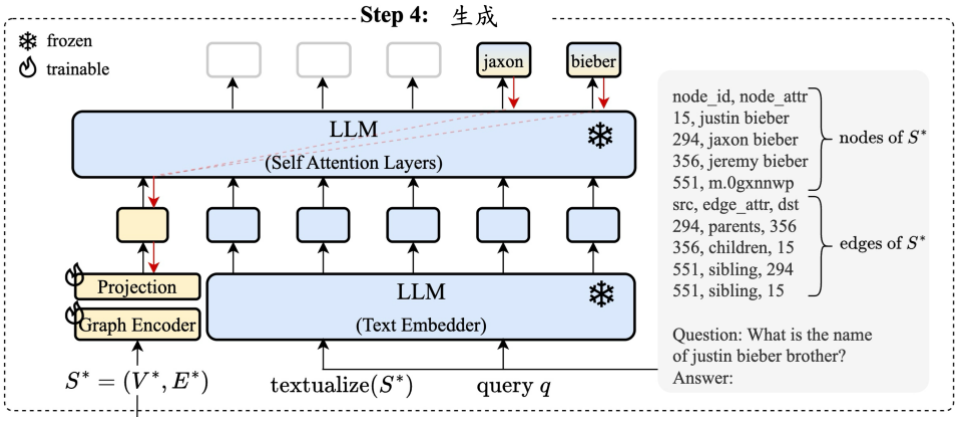


图 11　G-Retriever 模型结构示意图[68]（局部）

2.3.3　图结构与 LLM 联合建模

为实现语言模型与图神经网络的多层次协同建模，研究者提出通过跨架构交互机制深度融合 Transformer 编码层与 GNN 层，使二者在联合训练中实现双向增益效应。此类方法通过参数共享与梯度反传的协同优化，同步增强 LLM 的图语义解析能力与 GNN 的文本关联建模能力。

GreaseLM[53]架构通过设计了特定的前向传播层使 LM 和 GNN 之间通过特殊的交互 token（[INT]）和交互节点（Interaction Node）进行双向信息传递，从而集成了 Transformer 层和 GNN 层。该方法在每一层中不断更新语言表示，同时通过 GNN 处理知识图，允许两者的信息相互影响，以增强对图任务的推理和理解能力。这使得 GreaseLM 在问答任务上获得了较高的性能。ENGINE[54]在每一层的 LLM 中引入了一个轻量级且可调的 G-Ladder 模块，该模块通过消息传递机制集成结构信息。这使得每个 LLM 层的输出能够传递到相应的 G-Ladder 模块，从而增强节点表示，进而用于节点分类等下游任务。

2.4 本章小节

本章内容分为三个主要部分。首先，系统阐述图神经网络的基本概念、典型架构及其在代码分析领域的应用实践。其次，分析了预训练模型的技术原理与构建方法，重点讨论代码预训练的相关技术路线。最后，介绍了预训练模型与图神经网络的融合方法，具体包括三种技术范式：（1）纯语言模型驱动范式，通过自然语言编码间接利用模型的推理能力；（2）图特征前缀注入，将图神经网络生成的结构化表征作为模型输入前缀；（3）图结构与 LLM 联合建模，建立双向特征交互机制实现图结构的显式交互与建模。

第三章　基于图神经网络的漏洞检测框架

本章主要介绍本研究所提出的基于图神经网络的漏洞检测框架，主要包括代码属性图的自动化构建，基于树和图的层次化表征学习，以及端到端训练与代码漏洞检测。

3.1　基于图神经网络的漏洞检测框架整体架构

传统基于图神经网络的漏洞检测方法面临两个主要问题。在代码属性图处理方面，冗余节点和边（如中间变量声明与非关键控制分支）会引入噪声干扰，削弱关键节点间的语义关联。在图神经网络架构层面，浅层堆叠结构难以有效捕捉高阶依赖关系，而深层模型又容易引发过平滑问题。这导致模型对代码结构信息利用不足、漏洞相关语义特征提取不充分以及难以充分建模远距离节点依赖等问题。为解决现有方法的局限性，本工作提出基于图神经网络的检测框架。如图 12 所示，该框架通过代码属性图自动化构建、基于树和图的层次化表征学习、漏洞检测三个阶段实现代码特征建模与漏洞检测。

首先利用静态分析工具 Joern 生成初始代码属性图，该图整合了抽象语法树、控制流图和程序依赖图。针对异质图结构中冗余信息干扰关键语义的问题，本研究设计了简化算法，将抽象语法树的原子节点归约至所属语句节点，同时保留关键的控制依赖与数据依赖边。为充分捕获细粒度语义与全局依赖，本研究设计了层次化特征学习机制。该机制创新性地结合 Tree-LSTM 和图注意力网络，前者负责建模语句节点的 AST 子树以提取细粒度语法特征，后者则专注于语句间的控制、数据依赖关系的上下文建模。采用基于注意力机制的图池化方法，通过动态加权聚合节点特征保留关键语义，并进一步建模远距离节点间的依赖关系。最终，利用多层感知机分类器实现端到端的漏洞检测。

3.2　代码属性图的自动化构建

针对原始代码属性图中细粒度节点冗余与关键语义易丢失的问题，本研究提出了一种代码属性图构建算法（如算法 1 所示）对原始代码属性图进行化简。该算法通过层级节点归并与依赖边重构，消除非必要语法节点和边的干扰以降低图复杂度，同时保留漏洞敏感语义路径以确保关键信息完整性。

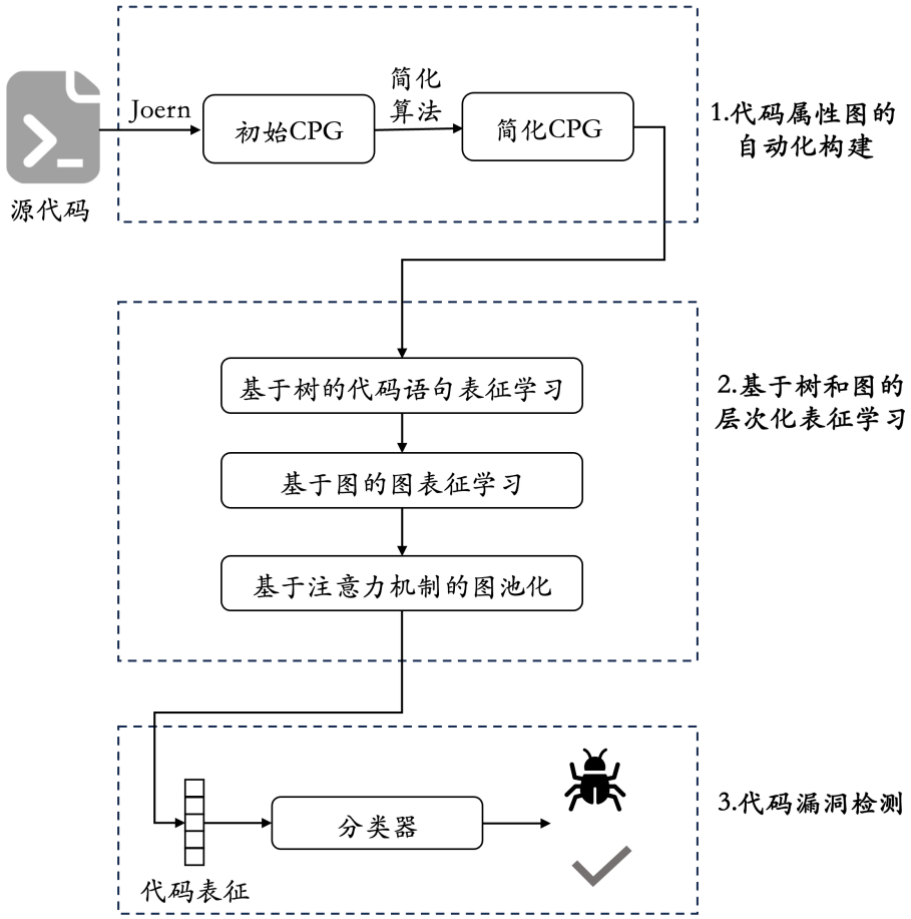


图 12　基于图神经网络的漏洞检测框架

具体实现上，算法将标识符和运算符等原子节点归并至所属语句节点，形成粗粒度语义单元，并将数据依赖、控制依赖与控制流中的细粒度依赖关系提升至语句层级，最终实现语义聚焦的图结构优化。算法输入为初始代码属性图 CPG，输出为优化后的 CPG'，下面详细说明该算法的三个核心处理阶段。

在语句节点识别阶段（第 7-13 行）中，算法首先从 CPG 中提取控制流图 CFG（第 7 行），随后遍历 CFG 中所有节点（第 8 行），采用基于代码行号的启发式规则进行节点分类。当某节点的所有后继节点均位于不同代码行时（第 9-10 行），判定为语句节点并加入集合（第 11 行）。该判定准则有效区分两种节点类型，语句节点包含函数入口（METHOD_RETURN）及代码行级 AST 根节点，承担程序逻辑单元的语义表征功能；其余节点作为非语句节点保留原子级语法细节。

在父节点映射阶段（第 14-22 行）中，为每个节点建立到最近语句父节点的映射。

对于语句节点直接建立自映射（第 17 行），而非语句节点通过广度优先搜索向上追溯最近的语句父节点（第 19 行）。

在边集合重构阶段（第 23-30 行）中，遍历原始边集合，仅对非 AST 边（包括 CFG、DDG、CDG 边）进行边重构（第 25 行），将边的源节点和目标节点分别替换为其对应的语句父节点（第 26-27 行），从而将控制依赖、数据依赖等语义关系提升至语句层级。而 AST 边保持原始连接（第 29 行），确保语法树结构的完整性。

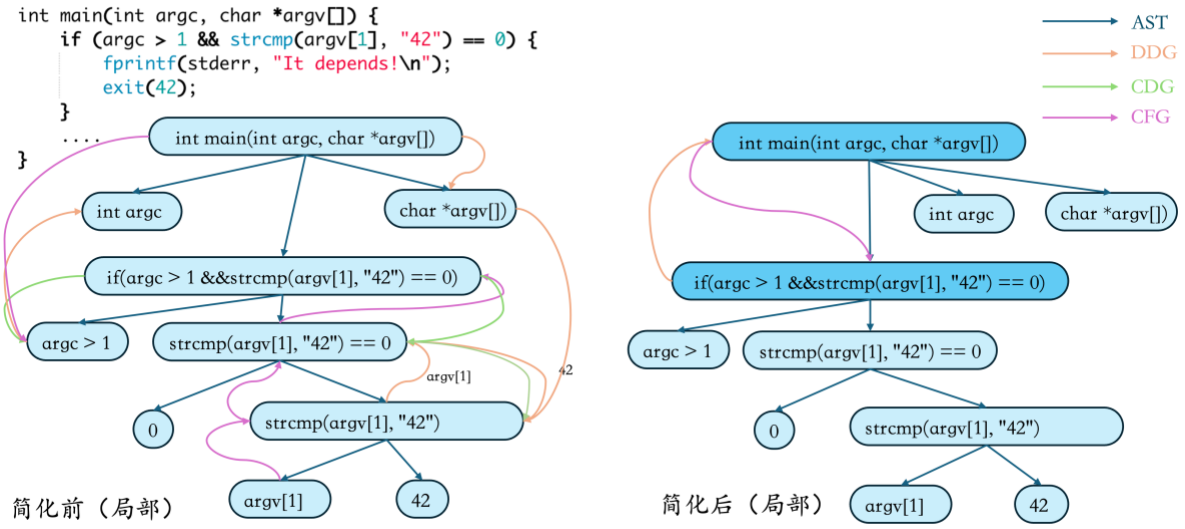


图 13　简化前后的代码属性图

经过上述处理，原始复杂的图结构被转换为具有层次结构的代码属性图（如图 13 所示），以支持神经网络的层次化表征学习。优化后的图节点分为两类：语句节点作为代码逻辑单元承载粗粒度语义，非语句节点作为抽象语法树叶子节点保留细粒度语法特征。边集合包含四类规范化依赖：AST 边（深蓝）连接语法单元，CFG 边（红）表征控制流顺序，DDG 边（橙）表示数据依赖，CDG 边（绿）表示控制依赖。该设计通过分层节点体系降低图复杂度以缓解异质信息干扰，同时保留跨语句依赖路径，使图神经网络能够捕获细粒度语义信息与语句级依赖关系。

算法 1 构建简化代码属性图

Algorithm 1 C++代码属性图简化

```
输入: 初始代码属性图 CPG
输出: 拓扑优化后的代码属性图 CPG'
初始化：Sstatement:= set()
        Sparent:= map()
        Sedge := set()
// 第一步: 计算 Statement 节点
CFG = Extract_CFG(CPG)
for node in CFG.nodes do
    successors := list(node.allSuccessors)
    if all(successor.line_number != node.line_number for successor in successors) then
        Sstatement.add(node)
    end
end
// 第二步: 计算父节点(Statement 类型)
for node in CPG.nodes do
    if node in Sstatement then
        Sparent[node] := node
    else
        parent := BFS to find closest statement node
        Sparent[node] := parent
    end
end
// 第三步: 将非 AST 边调整至 statement 节点之间
for edge in CPG.edges do
    if edge is not AST edge then
        u' := Sparent[edge.source]
        v' := Sparent[edge.target]
        Sedge.add((u', v', edge.type))
    end
end
// 第四步: 构建 CPG'
CPG' := Create empty graph
CPG'.add_nodes(CPG.nodes)
for (u', v', type) in Sedge do
    CPG'.add_edge(u', v', type)
end
Return CPG'
```

3.3　基于树和图的层次化表征学习

本方法采用层次化表征学习框架生成代码图表征：首先通过基于树的编码器提取细粒度语法特征，其次利用图注意力网络建模语句节点依赖关系，并通过图池化生成全局语义表征。下文将围绕这两个核心模块展开详细阐述。

3.3.1　基于树的代码语句表征学习

传统基于静态词嵌入的节点初始化方法在实际应用中面临词汇表外（Out-of-Vocabulary，OOV）问题，这主要源于用户自定义标识符（如 temp_buffer_42）的多样性和低频 API（如 openssl_encrypt）的稀疏性。由于有限训练语料难以全面覆盖这些特殊词汇，导致语义表征缺失。

为解决上述问题，本研究采用预训练模型（CodeT5+编码器）生成节点初始向量表征。具体而言，对于抽象语法树中的节点j，分别提取其类型属性（如<operator>.equals）与词法属性（如 strcmp），通过独立嵌入层获取类型向量$e_j^{type} \in \mathbb{R}^d$与词法向量$e_j^{\mathrm{lex}} \in \mathbb{R}^d$，将二者拼接作为节点初始表征：

$$x_j = \left[e_j^{type}, e_j^{lex}\right] \in \mathbb{R}^{2d} \tag{3.1}$$

其中通过类型向量捕获结构特征（如运算符、表达式类型等），词法向量保留具体标识符的语义信息（如 strcmp 的字符串比较语义）。

在代码语句表征学习阶段，本研究基于 Tree-LSTM 的递归编码器以捕捉 AST 的层级化语法特征。给定语句 s 对应的 AST 子树，采用自底向上的递归计算范式实现语法信息的门控式传播。对于每个语句节点 s 对应的抽象语法树子树，从叶子节点开始，自底向上地递归计算每个节点的嵌入表示，直至根节点。

具体而言，对于非叶节点 j，首先聚合所有子节点$k \in C(j)$)的隐藏状态h_k，以得到子节点特征之和：

$$\tilde{h}_j = \sum_{k \in C(j)} h_k \tag{3.2}$$

输入门i_j控制当前节点信息的吸收强度：

$$i_j = \sigma\left(W^{(i)} x_j + U^{(i)} \tilde{h}_j + b^{(i)}\right) \tag{3.3}$$

遗忘门f_{jk}调节子节点记忆的保留比例以防止梯度消失：

$$f_{jk} = \sigma(W^{(f)}x_j + U^{(f)}h_k + b^{(f)}) \tag{3.4}$$

输出门o_j决定状态输出比例：

$$o_j = \sigma(W^{(o)}x_j + U^{(o)}\tilde{h}_j + b^{(o)}) \tag{3.5}$$

记忆单元C_j的更新综合当前输入与子节点记忆：

$$u_j = tanh(W^{(u)}x_j + U^{(u)}\tilde{h}_j + b^{(u)}) \tag{3.6}$$

$$C_j = i_j \odot u_j + \sum_{k \in C(j)} f_{jk} \odot c_k \tag{3.7}$$

最终生成隐藏状态h_j：

$$h_j = o_j \odot tan\,h(c_j) \tag{3.8}$$

其中$\odot$表示逐元素乘法。该机制使模型能够自适应调节不同语法层级的信息流，例如增强 API 调用节点（如 strcmp）对其关键参数的关注，抑制无关语法元素的干扰。

上述计算过程通过深度整合抽象语法树的结构特性与深度学习模型的表征能力，实现了代码语句的多层次表征学习。以图 14 所示的 strcmp(argv[1], '42') == 0 的 AST 子树为例：根节点 n1 作为逻辑运算符节点(<operator>.equals)，通过两个子节点分别连接右操作数（LITERAL 类型节点 0）与左操作数（API 调用节点 strcmp）。左操作数节点 n3 进一步展开为函数参数子树，其中节点 n4 对应字符串字面量'42'，节点 n5 关联变量标识符 argv[1]，而 n5 的子节点 n6（LITERAL 类型）和 n7（IDENTIFIER 类型）分别表征参数值与其容器变量。对其中的语句节点进行表征学习，首先进行节点向量初始化，通过 CodeT5+分别编码各节点的类型与词法属性，生成融合语法类型与语义细节的初始向量。然后，基于 Tree-LSTM 的递归计算从叶节点（如 n6 和 n7）自底向上传播特征，其中非叶节点 n5 通过公式(3.2)聚合子节点嵌入，并利用公式(3.3)-(3.8)的门控机制动态调节信息流，输入门强化关键参数（如 argv[1]）的语义融合，遗忘门抑制无关语法元素的干扰。最终，根节点 n1 通过记忆单元融合左右操作数的语法语义特征，输出语句级表示作为局部上下文感知的编码结果。

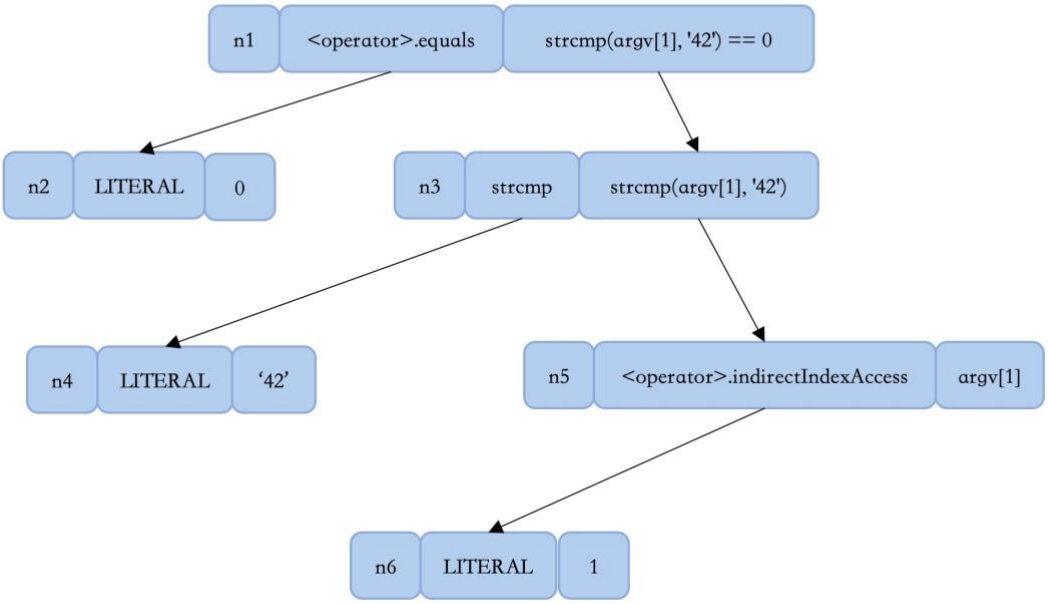


图 14　抽象语法树

3.3.2　基于图的图表征学习

图表征学习通过图节点上下文建模与图池化两个阶段实现全局表征的学习。图节点上下文建模采用消息传递和聚合机制捕获节点的依赖关系，而图池化则从全局视角整合图节点的信息，以用于漏洞检测。

3.3.2.1　图表征学习

本研究采用图注意力网络作为基础模型架构，其主要思想在于在图的节点间引入注意力机制。与传统图神经网络不同，该方法允许节点在更新自身特征时，根据邻居节点的重要性进行动态加权聚合。具体而言，在节点特征更新过程中，该方法首先计算目标节点与其每个邻居节点的相对重要性，然后基于计算得到的注意力系数进行有区别的信息聚合，从而有效提升节点表征的区分度和表达能力。

具体实现过程如下：对于给定的节点i和其邻居节点$j \in N_i$，首先，通过共享权重矩阵 W 对节点特征进行线性变换，然后利用可学习向量 a 计算原始注意力得分。

$$e_{ij} = a([W\,\mathrm{h}_i \,||\, Wh_j]), j \in N_i \tag{3.9}$$

然后使用 LeakyReLU 激活函数处理 e_{ij}，并除以节点i的所有邻居的权重和，得到归一化后的注意力系数。

$$\alpha_{ij} = \frac{exp(LeakyReLU(e_{ij}))}{\sum_{k \in N_i} exp(LeakyReLU(e_{ik}))} \tag{3.10}$$

根据(3.10)式计算得到的归一化注意力系数，对所有邻居节点加权求和。此外，为了增强模型的表达能力和泛化性，使用多头注意力机制，对 K 个注意力头的结果进行拼接得到节点i的最终表征。

$$h_i' = \sigma(\sum_{j \in N_i} \alpha_{ij} W) \tag{3.11}$$

$$h_i'(K) = \mathop{\|}_{k=1}^{K} \sigma(\sum_{j \in N_i} \alpha_{ij}^k W^k) \tag{3.12}$$

设 GAT 层数为l，则 GAT 层后的节点向量表征为:$h_1^l, h_2^l, \dots, h_n^l$，其中$n$为图中的节点数量。这种基于注意力机制的图表征学习方法，能够有效捕捉图中节点间的复杂依赖关系，提升下游任务的性能表现。

3.3.2.2 图池化

通过邻域聚合建模节点上下文特征后，需通过池化操作有效整合节点特征，以生成全局图表征。现有池化方法可分为平面池化与层次池化两类，前者采用算术运算（如节点嵌入均值或求和）生成全局表征，后者通过节点聚类或丢弃策略实现图结构层级粗化。然而，节点聚类池化存在较高的计算与空间复杂度，而节点丢弃池化虽提升计算效率却易导致信息损失。当前主流方法多采用非学习的全局均值池化、最大值池化，或通过虚拟节点与全图节点连接的嵌入聚合机制来捕获全局信息。

在漏洞检测任务中，漏洞触发点通常仅涉及少量代码节点，且这些关键节点可能相距很远。浅层图神经网络受限于局部邻域聚合机制，其 K 层网络仅能捕获 K 跳邻域信息，当关键节点间距超过网络深度时，跨语句依赖关系将无法有效传递。为此，本研究在图池化阶段引入 Transformer 架构，通过多头自注意力机制建立全图节点的动态交互。如图 3 所示，Transformer 摒弃了图神经网络的拓扑结构约束，允许任意节点间计算注意力权重，其全局信息聚合能力在自然语言处理的远距离语义建模中已得到验证。该架构在保留图神经网络局部结构特征提取能力的同时，通过全局注意力机制突破传统池化方法的拓扑约束，从而避免启发式节点选择导致的关键语义丢失。

如图 15 所示，Transformer 通过注意力机制捕获远程依赖关系，其本质可视为动态构建的全连接图结构。具体实现包含以下几个步骤：

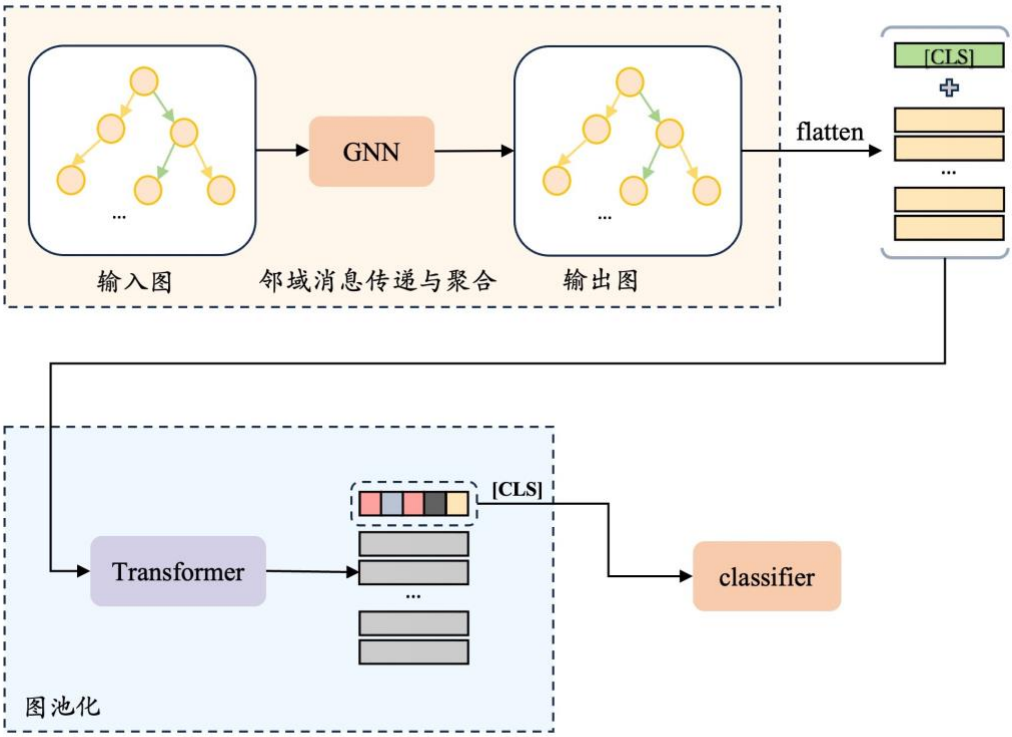


图 15　基于图的表征学习

首先，将经过多层消息传递生成的图节点按照代码顺序展开为节点嵌入序列$\{h_1^l, h_2^l, \dots, h_n^l\}$，并在序列前加入[CLS]标记作为 Transformer 模块的输入。

随后，进行特征空间对齐，通过线性投影矩阵$W_{proj} \in \mathbb{R}^{d \times d_g}$将$d_g$维图表征映射至 Transformer 的$d$维空间，并采用层归一化处理：

$$h_v^0 = LayerNorm\left(W_{proj} h_v^l\right) \tag{3.13}$$

其中$W_{\mathrm{Proj}} \in \mathbb{R}^{d_t \times d_g}$，$d_t$和$d_g$分别是 Transformer 维度和最终 GNN 嵌入的维度。

构建包括 L 层的标准 Transformer 编码器，每层由多头自注意力机制和前馈网络组成，节点间关联强度计算为：

$$a_{v,u}^l = \left(W_Q^l h_v^{l-1}\right)^\top \left(W_K^l h_u^{l-1}\right) / \sqrt{d_t} \tag{3.14}$$

经 softmax 归一化后生成注意力分布：

$$\alpha_{v,u}^l = \mathrm{softmax}\left(\sum_{w \in V} a_{v,w}^l\right) \tag{3.15}$$

节点表征更新为邻域加权聚合：

$$h_v^l = \sum_{w \in V} \alpha_{v,w}^l W_V^l h_w^{l-1} \tag{3.16}$$

其中W_Q^l，W_K^l和W_V^l分别是第l层单个注意力头的学习查询、键和值矩阵。

值得注意的是，考虑到图神经网络已在层级传播中编码拓扑信息，本设计主动摒弃位置嵌入以强化语义关联的拓扑无关性。经过若干层 transformer 的处理后，通过注意力模块计算[CLS]词元和代码节点嵌入之间的关系，有区分性地聚合与漏洞相关的语义信息，提取出最后一层[CLS]位置对应的向量表征，作为最终的图表征。

$$h_{graph} = h_{cls}^n \tag{3.17}$$

3.4 端到端训练与代码漏洞检测

通过层次化的表征学习得到图表征h_{graph}后，本研究设计了两层多层感知机实现分类决策，其可形式化为：

$$o = softmax\big(\sigma\big(W_1 h_{graph}\big)W_2\big), predict = argmax(o) \tag{3.18}$$

其中$\mathrm{W}_1 \in \mathbb{R}^{d \times d_h}$和$\mathrm{W}_2 \in \mathbb{R}^{d_h \times 1}$是两个可学习的权重矩阵，$\sigma$是一个激活函数，$predict \in \{0, 1\}$指示代码是否存在漏洞。

为进一步提升模型训练效率与性能，本研究提出多任务联合训练框架，交叉熵损失直接监督分类器输出，驱动表征学习组件生成具有判别性的特征表示；三元组损失通过约束样本间距，优化特征空间分布，确保同类样本在特征空间中紧密聚集而不同类样本相互疏离；焦点损失则通过动态调整样本权重，抑制易分类样本的梯度贡献，同时增强对难分类样本的优化，有效缓解数据分布不平衡问题。后续小节将详细阐述各损失函数的设计原理与数学形式化表示。

3.4.1 交叉熵损失

代码漏洞检测本质上是一个二分类任务，其目标是判断给定代码片段是否包含漏洞。设模型输出为漏洞存在的概率估计值 $\hat{y} \in [0,1]$，真实标签为$y \in \{0,1\}$（其中 1 表示存在漏洞，0 表示不存在漏洞），则交叉熵损失（Cross-Entropy Loss）函数可形式化定义为：

$$\mathcal{L}_{\mathrm{CE}} = -[y \log \hat{y} + (1-y)\log(1-\hat{y})] \tag{3.19}$$

其中N为样本数量，y_i为第i个样本的真实标签，$\hat{y}_i$为第i个样本的预测概率。模型的训练过程通过最小化交叉熵损失函数来优化权重矩阵。

该损失函数通过最小化模型预测分布与真实分布之间的差异，促使模型有效学习区分漏洞样本与非漏洞样本的判别性特征。

3.4.2　焦点损失

在基于深度学习的漏洞检测任务中，经验风险最小化范式常因数据分布偏斜导致模型偏差。具体而言，当正样本（有漏洞代码）与负样本（无漏洞代码）呈现显著数量级差异时，常规交叉熵损失函数会因以下两个问题影响模型性能：其一，多数类的累积梯度主导参数更新方向；其二，易分类样本（如多数类）的损失贡献度掩盖了关键少数类的学习信号。这种偏差效应最终将导致分类器决策边界向多数类（无漏洞样本）偏移，从而削弱模型对关键漏洞特征的判别能力。

针对上述问题，本研究采用 Lin 等人[60]提出的焦点损失（Focal Loss）作为核心优化目标。该损失函数通过双参数机制重构损失空间，在标准交叉熵损失的基础上引入难易样本调节因子（γ）与类别平衡因子（α_t），其数学表达式可形式化定义为：

$$\mathcal{L}_{\text{Focal}} = -\alpha_t(1-\hat{y}_t)^{\gamma}\log(\hat{y}_t)) \tag{3.20}$$

$$\hat{y}_t = \begin{cases} \hat{y}, if t = 1(\text{正类}) \\ 1-\hat{y}, if t = 0(\text{负类}) \end{cases} \tag{3.21}$$

各参数具有明确的数学物理意义。其中，当样本属于正类时（$t=1$），$\hat{y}_t = \hat{y}$；当样本属于负类时（$t=0$），$\hat{y}_t = 1-\hat{y}$。类别平衡因子$\alpha_t \in [0,1]$，基于先验类别频率设置，用于补偿正负样本的数量级差异。在漏洞检测场景中，可通过增大负样本的α_t值来强化少数类的影响力。指数调制因子$\gamma \geq 0$，控制困难样本挖掘强度，易分类样本（即预测置信度$\hat{y}_t$趋近于 1 的样本）的损失贡献度随着$(1-\hat{y}_t)^{\gamma}$呈指数衰减，促使模型聚焦于预测概率接近决策边界的困难样本。

值得关注的是，该函数的调节机制在漏洞检测任务中展现出特殊优势。对于高频出现的非漏洞样本，其较高的易分类特性使得$(1-\hat{y}_t)^{\gamma}$项产生显著的损失衰减效应；而稀疏的漏洞样本则通过α_t项获得损失增强。这种动态平衡机制有效优化了模型的决策边界，提升了对关键少数类的识别精度。

3.4.3　三元组损失

本研究采用三元组损失（Triplet Loss）优化代码表征学习，以增强特征空间的类内聚合性和类间分离性。该损失函数通过最小化同类样本间的特征距离，同时最大化异类

样本间的特征距离，从而提升模型对语义相似但标签相异样本的判别能力。在漏洞检测任务中，该方法能有效区分边界模糊的漏洞样本与非漏洞样本。

三元组损失基于样本三元组构建特征空间优化目标，一个三元组由锚点（anchor）、正样本（positive sample）和负样本（negative sample）组成。其中，锚点样本（a）与正样本（p）同属一个类别，与负样本（n）分属不同类别。其核心优化目标是让锚点和正样本的距离尽可能小，而锚点和负样本的距离尽可能大，可形式化为：

$$\mathcal{L}_{\text{Triplet}} = \max(0, d(a,p) - d(a,n) + \alpha) \tag{3.22}$$

其中，$d(\cdot\ ,\ \cdot)$表示特征向量间的距离度量，$d(a,p)$表示锚点和正样本之间的距离，$d(a,n)$表示锚点和负样本之间的距离。α为预设的边界超参数（margin），用于约束正样本对与负样本对的最小间隔距离。该损失函数通过最小化$d(a,p) - d(a,n)$的差值，使得模型满足$d(a,p) + \alpha < d(a,n)$的相对约束。这种约束机制不仅增强了同类特征的紧致性，还通过预设的边界条件在异类样本间建立明确的分隔带，从而提升分类决策面的健壮性。

要实现健壮的深度学习过程，训练数据十分重要。本研究提出基于语义相似度的三元组构建策略，具体过程如图 16 所示。首先，利用预训练的 CodeT5+模型对全部训练代码片段进行深度语义编码，生成维度为 256 的向量表征空间。根据漏洞检测标签将样本划分为正样本库D^+（漏洞样本集合），负样本库D^-（无漏洞集合）。对于每个锚点样本$x_i \in D^+$，以其编码向量作为查询向量，使用 Faiss 工具进行近似最近邻搜索，在同类库D^+中检索余弦相似度最高的 top-k 个有漏洞样本构成候选集P_i，同时在异类库D^-中筛选余弦相似度最高的 top-k 个无漏洞样本构成候选集N_i。通过均匀随机采样机制从P_i和N_i中分别选取一个正样本p_i和负样本n_i，最终构建具有强语义关联的三元组(x_i, p_i, n_i)。为维持数据分布的对称性，对D^-中的样本执行镜像操作。以无漏洞样本为锚点时，从同类库D^-中检索语义最相近的样本构成无漏洞样本候选集，从异类库D^+中检索与锚点语义相似度最高的样本构成有漏洞样本候选集。在联合训练框架中，模型的总损失定义为交叉熵损失和三元组损失的加权和：

$$\mathcal{L}_{\text{total}} = \lambda_{\text{CE}}\mathcal{L}_{\text{CE}} + \lambda_{\text{Triplet}}\mathcal{L}_{\text{Triplet}} \tag{3.23}$$

其中，超参数 $\lambda_{\text{CE}}, \lambda_{\text{Triplet}} \in [0,1]$ 分别控制两类损失的贡献权重，且满足约束条件 $\lambda_{\text{CE}} + \lambda_{\text{Triplet}} = 1$。在本研究所提出的图结构与预训练模型融合框架中，通过图神经网络

从代码属性图中学习结构化拓扑特征，利用预训练的文本编码器通过自注意力机制捕获代码片段的上下文语义信息。两类特征经融合后，生成统一的代码表征向量。该表征承担双重优化目标（1）分类任务监督：输入分类器进行漏洞类别预测，通过交叉熵损失$\mathcal{L}_{\mathrm{CE}}$优化分类决策边界；（2）度量学习约束：作为三元组损失$\mathcal{L}_{\mathrm{Triplet}}$的计算输入，约束同类与异类样本在特征空间中的相对距离。

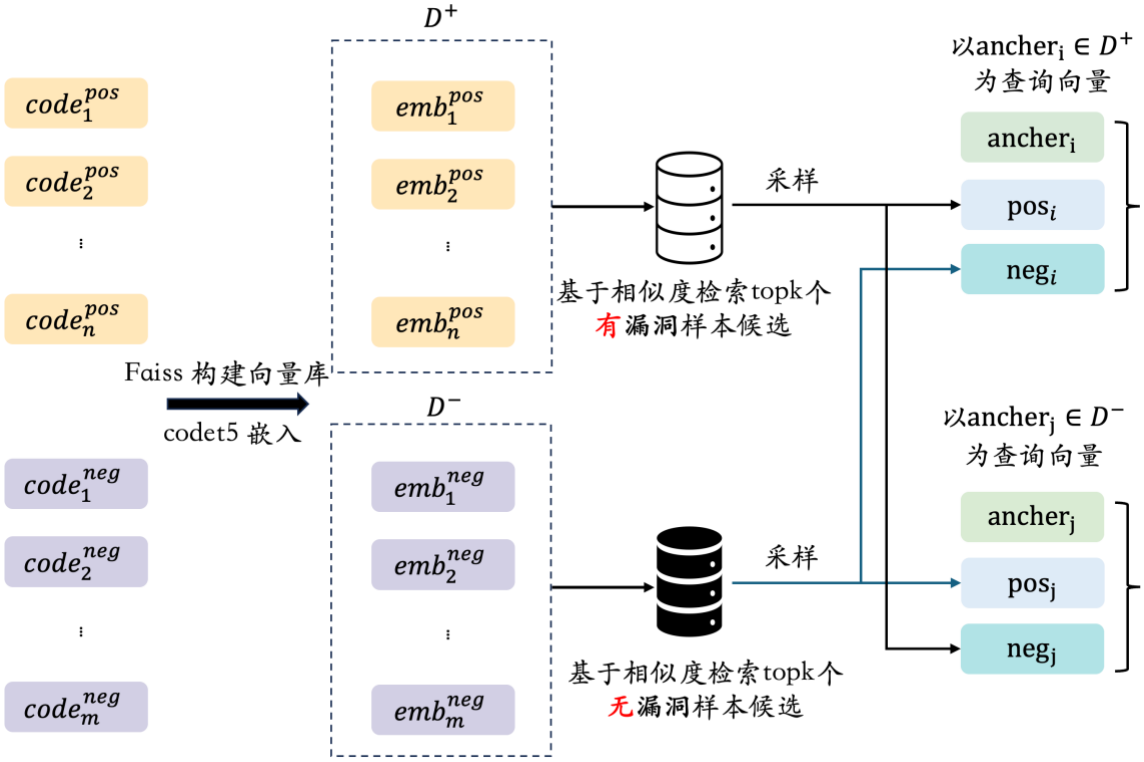


图 16　三元组训练数据集的构建过程

该联合优化机制采用多任务学习策略来增强特征空间。交叉熵损失主要负责提升表征向量的类别区分能力，三元组损失则着重优化类内样本的聚集程度和类间样本的分离程度。这两种损失函数相互配合，在优化分类性能的同时，改善特征空间的判别性结构。通过重点选择类间边界区域的困难样本，特别是那些语义相似但标签不同的样本对，该策略帮助模型捕捉更细微的判别特征。这种方法有效降低了随机采样引入的训练噪声，从而显著提升了模型在分类任务上的表现和泛化能力。

3.5　本章小节

本章针对代码漏洞检测任务中源代码表征学习的关键挑战，提出了一种基于图神经

网络的漏洞检测框架。在特征预处理阶段，该框架通过图简化算法消除冗余信息，同时保留控制依赖与数据依赖等关键语义边，从而构建利于模型学习的代码属性图。在特征学习阶段，采用 Tree-LSTM 对代码语句的语法子树进行细粒度特征提取，以捕获局部语句特征；在此基础上，结合图注意力网络实现邻域语义传播，以建模语句之间的依赖关系；最后利用基于 Transformer 的图池化方法完成全局特征的聚合，并进一步捕获节点长距离依赖。在训练阶段，为提高模型训练效率与泛化性，采用联合优化策略。交叉熵损失函数驱动漏洞分类边界学习，强化判别性特征提取。三元组损失约束特征空间，通过距离度量学习实现类内紧致性与类间分离性。焦点损失动态调整样本梯度贡献权重，通过困难样本聚焦机制缓解类别不平衡问题。

第四章　图结构与预训练模型融合的漏洞检测框架

本章主要介绍图结构与预训练模型融合的漏洞检测框架所涉及的算法设计和实现，包括图结构表征学习、图结构与预训练模型融合以及端到端训练与代码漏洞检测。

4.1　图结构与预训练模型融合的漏洞检测框架整体架构

近年来，基于 Transformer 架构的预训练语言模型在自然语言处理领域取得突破性进展，其通过大规模预训练获得的语义理解与零样本迁移能力，显著提升了机器翻译、文本理解和文本生成等任务的性能水平。然而，当应用于代码文本处理时，传统基于词元粒度的序列建模方法难以有效捕捉程序语言特有的结构化语义特征。以代码漏洞检测任务为例，代码依赖图能够显式表征语句间的逻辑依赖关系，而基于序列建模的方法由于缺乏结构建模能力，往往难以识别此类模式。这种结构信息的缺失直接影响漏洞检测效果，因为潜在漏洞常隐藏在跨语句的控制依赖或数据依赖关系中。

图神经网络在代码结构建模领域展现出独特优势，其消息传递机制可直接建模程序的控制依赖与数据依赖等关系，通过迭代聚合邻域节点能够有效捕捉多跳语义关联。但现有研究面临双重挑战：在结构建模方面，传统图神经网络受限于局部邻域聚合机制，浅层网络难以捕获远距离依赖，深层网络又易引发节点特征过平滑现象[38]；在语义理解方面，尽管代码中的变量命名和函数调用（如 strcpy 通常暗示缓冲区溢出风险）包含重要语义信息，但传统方法使用的通用文本嵌入模型 Word2Vec 难以有效学习代码特有的语义特征，导致节点表征不够充分。

为解决上述问题，研究者尝试结合图结构与预训练模型的优势，目前主流技术路径可分为两类。第一类方法通过人工设计的提示模板将图结构转换为自然语言指令，利用大语言模型的推理能力进行漏洞检测。这种方法在结构转换过程中容易丢失拓扑信息，同时受到输入长度限制。第二类方法则采用冻结参数的预训练模型生成节点初始表征，节点虽然继承了语义理解优势，并会通过图神经网络的传播机制进行局部调整，但文本编码器无法针对当前图结构进行端到端的语义适应。两类方法均未有效解决核心矛盾，如何在保留图结构完整性的同时，实现文本语义与拓扑特征的深度融合。

针对现有方法的局限性，本研究提出一种图结构与预训练语言模型融合的漏洞检测框架 GNNAdapter。该框架的核心创新之处在于同时学习代码的结构特征与语义特征，

即从代码属性图提取的拓扑结构特征和由预训练模型生成的文本语义特征。前者捕捉程序的控制依赖和数据依赖等关系，后者编码变量命名和 API 调用等语义信息。通过设计专门的融合机制，该框架实现了两类特征的互补增强，有效突破了传统单一表征方法的局限性。具体实现上，首先利用预训练语言模型对代码属性图节点进行语义初始化，包括变量名和函数名等代码元素的嵌入表征。然后通过图神经网络聚合语法结构特征，涵盖语句内语法模式及语句间控制流和数据流依赖。在关键的特征融合阶段，采用图结构感知的浅层拼接或图结构引导的注意力深度融合策略，最终生成结构特征与语义特征有效结合的增强型联合代码表征。

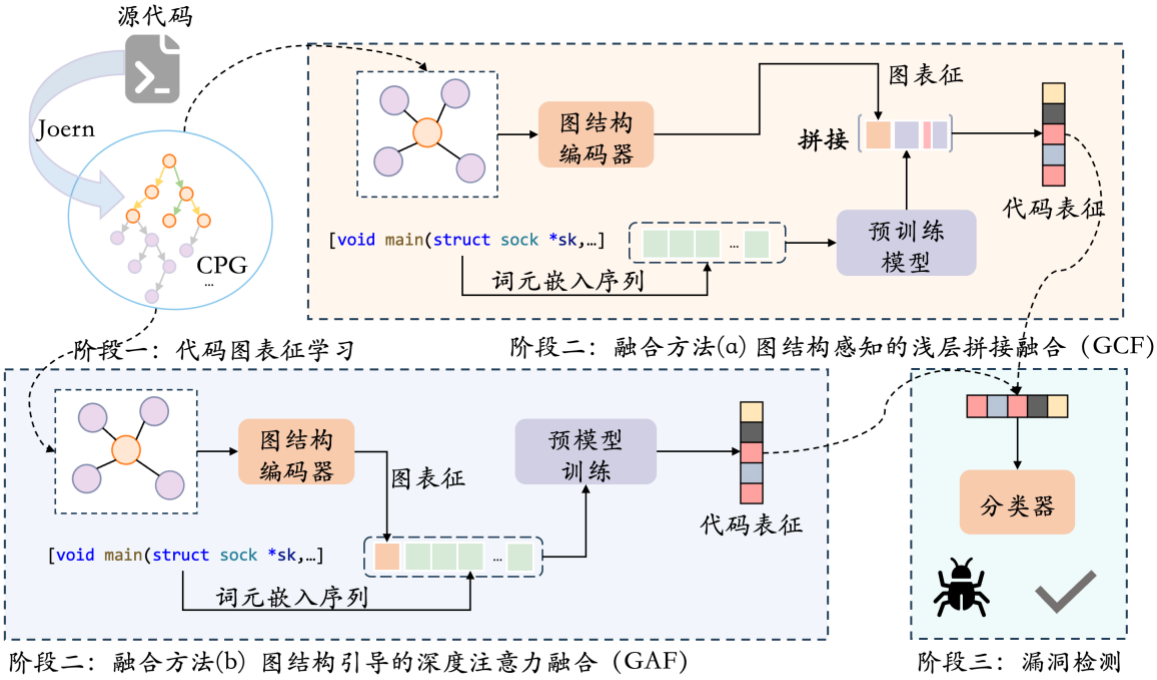


图 17　图结构与预训练模型融合的漏洞检测框架

如图 17 所示，具体实现流程分为三个阶段：（1）图结构表征学习阶段：以代码属性图作为输入，通过 GNN 提取代码语句内部的语法结构特征以及语句间的控制依赖和数据依赖关系，生成高质量的图表征；（2）图结构与代码文本的特征融合阶段：采用图结构感知的浅层拼接融合或图结构引导的深度注意力融合策略，将提取的代码结构表征与文本语义表征进行整合，获得融合后的增强代码表征；（3）漏洞检测阶段：利用多层感知机分类器对代码表征进行二分类，实现漏洞存在与否的判断。该框架通过多层次特征提取与融合机制，有效解决了传统方法在代码语法和语义理解方面的不足，为软件漏洞检测提供了新的解决方案。

4.2　图结构与预训练模型融合的表征学习

4.2.1　图结构表征学习

图表征学习阶段（详见第 3 章）采用基于树和图的层次化表征学习框架，充分建模代码语法结构信息。具体而言，对于一段源代码函数$code_i$，首先基于 Joern 静态分析工具构建原始 CPG，然后应用本研究提出的图结构简化算法进行拓扑优化，通过去除冗余噪声和合并语义等价边，最终生成具有层次结构的精简 CPG。在优化后的图结构中，每个代码语句对应独立的抽象语法树，语句间通过控制依赖和数据依赖建立逻辑关联。

表征学习阶段主要包含四个关键步骤：（1）语义感知初始化：利用预训练语言模型对代码结构图中的节点进行初始嵌入，将变量名、函数名等代码元素的文本语义映射至连续向量空间；（2）语法上下文聚合：基于每个语句节点的 AST，采用树状长短期记忆网络自底向上聚合节点信息，生成融合语法上下文的语句表征；（3）结构关系建模：利用图注意力网络分析语句节点间的控制依赖和数据依赖等关系，通过多头注意力机制计算跨语句的语义关联权重，实现跨语句语义关系的自适应表征学习；（4）图池化处理：设计基于注意力机制的图池化层，通过节点特征的加权聚合，生成融合局部语法特征与全局结构关系的图表征x_i。

为实现图表征x_i与预训练模型的语义空间对齐，框架设计了一个可学习的投影模块。该模块通过参数化的线性变换，将图结构表征x_i映射至预训练模型的嵌入空间，得到对齐后的表征z_i。该过程可形式化表示为以下公式：

$$x_i = GNN\left(x_i^0, X, A|\Theta_g\right) \tag{4.1}$$

$$z_i = W_p x_i + b_p \tag{4.2}$$

其中x_i^0和A分别代表 GNN 的初始节点特征和邻接矩阵，Θ_g为 GNN 的可学习参数，矩阵W_p和偏置项b_p为投影模块的可学习参数。这一设计使得 GNN 捕获的程序控制依赖与数据依赖等拓扑特征能够与预训练模型编码的文本语义特征在统一空间中进行融合，并通过端到端训练机制实现异构特征的协同优化，为漏洞检测任务构建感知结构特征的联合表征空间。

4.2.2　图结构与代码文本的特征融合

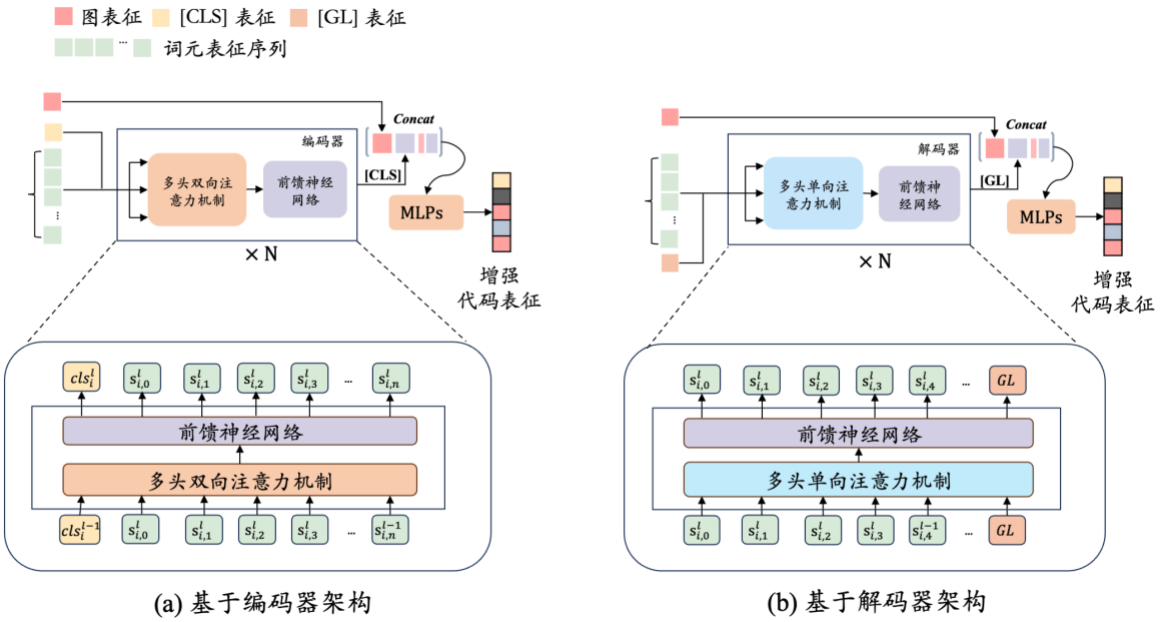
在生成代码结构表征z_i后，需将其与代码文本语义表征进行特征融合，以共同建模

代码的结构特征与语义特征。本研究提出两种融合范式：（1）图结构感知的浅层拼接融合（Graph-Aware Shallow Concatenation Fusion, GCF），其在特征空间通过浅层拼接实现代码结构图表征和代码文本表征的静态融合。（2）图结构引导的深度注意力融合（Graph-Guided Deep Attention Fusion, GAF），其通过注意力机制实现代码图结构表征与代码词元表征的深度动态融合。

4.2.2.1 图结构感知的浅层拼接融合

如图 18（a）所示，图结构感知的浅层拼接融合通过解耦式特征提取与静态拼接，实现代码语法与语义的联合表征学习。对于函数级代码片段，该方法首先采用 CodeT5+ 编码器提取代码文本的全局语义表征$s_i \in \mathbb{R}^d$，其聚合了自注意力机制捕获的全局上下文信息。同时，图神经网络对代码属性图进行编码，生成结构表征$z_i \in \mathbb{R}^d$，建模控制依赖与数据依赖的拓扑依赖关系。在特征集成阶段，语义表征$s_i \in \mathbb{R}^d$和结构表征$z_i \in R^d$通过拼接与非线性变换实现融合，生成联合表征h_{final}。该过程可形式化表示为以下公式：

$$h_{final} = MLP\big(CONCAT(s_i, z_i)\big) \tag{4.3}$$



(a) 基于编码器架构　　(b) 基于解码器架构

图 18　图结构感知的浅层拼接融合

在解码器模型（如 phi-1）中，自回归生成机制通过单向注意力掩码确保模型仅能基于历史上下文预测下一个词元。为有效捕获全局上下文信息，可在输入序列末端引入特殊标记[GL]作为全局上下文的聚合标记（如图 18（b）所示）。在序列生成过程中，解

码器通过多头注意力机制更新[GL]位置的隐状态$h_{GL} \in \mathbb{R}^d$，使其累积前序所有词元的语义信息。最终，将h_{GL}作为代码语义表征，将其与图结构表征z_i进行特征拼接融合，构建具有全局结构与语义感知的联合表征h_{final}。

4.2.2.2　图结构引导的深度注意力融合

图结构引导的深度注意力融合方法通过将代码结构表征z_i作为软提示（Soft Prompt）注入编码器的注意力计算过程，实现结构特征与语义特征的动态联合表征学习，具体实现分为三个阶段，如图 19（a）所示。

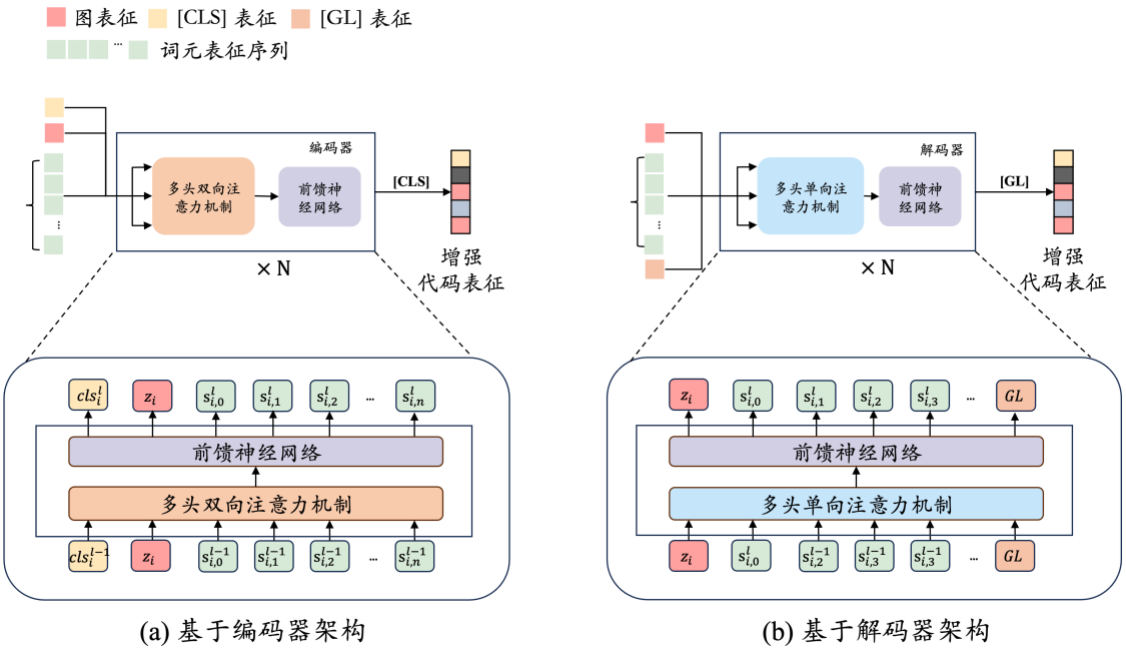


图 19　图结构引导的深度注意力融合

第一步进行代码文本的分词与嵌入，首先通过预训练模型的分词器（Tokenizer）将源代码解析为词元序列：

$$\{t_{i,0},\ldots,t_{i,k}\} = Tokenizer(c_{i,0},\ldots,c_{i,k}) \tag{4.4}$$

并通过嵌入层将离散词元映射为连续向量以捕捉变量名、API 调用等代码的细粒度语义信息，得到词元嵌入序列：

$$\{s_{i,0},\ldots,s_{i,k}\} = Embedding_Layer(t_{i,0},\ldots,t_{i,k}) \tag{4.5}$$

第二步进行输入构造。将代码图表征z_i拼接至代码词元表征序列之前，并在序列前端添加 [CLS] 标记，作为全局上下文的聚合器，形成编码器的输入：

$$X_{input} = \{[CLS], z_i, s_{i,0},\ldots,s_{i,k}\}$$

第三步进行联合编码与表征提取。将X_{input}输入至编码器，通过多层编码器块的多头自注意力机制，图结构特征与文本语义特征进行深度交互，最终提取 [CLS] 标记的隐藏状态作为融合表征，整个过程可以形式化如下：

$$h_{CLS} = LLM_{encoder}\left([CLS], z_i, s_{i,0}, \dots, s_{i,k} \mid \Theta_g, \Theta_{LLM}\right) \tag{4.6}$$

该方法同样可以泛化到解码器模型，主要调整在于输入序列构造，如图 19（b）所示。将图嵌入z_i拼接至词元嵌入序列之前，并在末端添加 [GL] 标记作为全局上下文的聚合器：

$$X_{input}^{Dec} = z_i, s_{i,0}, \dots, s_{i,k}, [\mathrm{GL}] \tag{4.7}$$

接着，通过自回归解码器的多层单向注意力机制，提取 [GL] 标记的隐藏状态作为增强代码表征：

$$h_{GL} = LLM_{decoder}\left(z_i, s_{i,0}, \dots, s_{i,k}, [GL] \mid \Theta_g, \Theta_{LLM}\right) \tag{4.8}$$

图结构引导的深度注意力融合机制的核心在于将代码图结构表征z_i深度嵌入 Transformer 的多层注意力计算过程。具体而言，由 GNN 生成的结构特征作为动态引导信号，通过可学习的注意力调节机制对词元的注意力分布进行重加权，从而引导模型聚焦于与漏洞检测相关的关键语义特征。相较于静态特征拼接方法，本方法通过将结构特征直接整合到自注意力计算环节，实现了结构信息对语义解析的动态引导。

4.3 端到端训练与代码漏洞检测

通过图结构与预训练模型融合框架学习得到代码表征$h_{final} \in \mathbb{R}^d$后，将其输入分类器进行漏洞概率预测。具体而言，分类器采用轻量级多层感知机，通过非线性变换实现从特征空间到概率空间的映射：

$$\hat{y} = \sigma\left(W_2 \cdot \sigma\left(W_1 \cdot h_{final}\right)\right) \tag{4.9}$$

其中h_{final}表示融合后的代码表征，$W_1 \in \mathbb{R}^{d \times d_h}$和$W_2 \in \mathbb{R}^{d_h \times 1}$为可训练的权重矩阵，$\sigma$为非线性激活函数，$\hat{y} \in [0,1]$表示漏洞存在的预测概率。具体而言，首先将代码表征$h_{final}$经$W_1$线性投影和激活函数处理，生成中间表征；随后通过第二层权重矩阵W_2得到$o_{logits} = \sigma\left(W_1 h_{graph}\right) W_2$，最终经 sigmoid 函数压缩至[0,1]概率区间。

为提升模型整体性能，本研究采用多任务联合训练策略。具体而言，该框架通过交

叉熵损失函数优化分类决策边界，促使模型学习更具判别性的特征表示；同时引入三元组损失在特征空间施加度量约束，实现类内样本聚合与类间样本分离；针对数据不平衡问题，采用焦点损失动态调整样本权重，通过抑制易分类样本的梯度贡献，使模型聚焦于难分类样本的学习。通过端到端的联合训练范式，该框架实现了图结构特征提取模块、预训练语义编码器与分类决策模块的协同优化，从而显著提升了漏洞检测任务的分类性能。具体实现详见 3.4 节。

4.4　本章小节

本研究针对预训练模型在代码漏洞检测中存在的结构信息缺失问题，提出了一种图结构与预训练模型融合的框架。该框架能够同时建模代码结构特征与语义上下文特征。在语义建模层面，基于预训练语言模型对代码属性图中的变量命名和函数调用等节点进行细粒度语义编码，从而捕获词法层面的深层语义信息。在结构建模维度，通过图神经网络建模控制依赖与数据依赖等关系，构建具有语法感知能力的结构化表征。框架包含两种异构特征融合方式。图结构感知的浅层拼接融合通过特征向量拼接实现结构表征与文本表征的静态融合，保持两类特征的独立空间分布。图结构引导的深度注意力融合则通过注意力层实现特征层面的深度交互。

第五章　实验结果与分析

本章通过实验验证了本文提出的两种漏洞检测方法的有效性。5.1 节对基于图神经网络的漏洞检测方法进行了实验分析与讨论，5.2 节则对图结构与预训练模型融合的漏洞检测方法进行了实验分析与讨论。实验采用标准数据集和统一评估指标，对比分析了不同方法的性能表现，并通过消融实验验证了关键技术的贡献。研究结果为基于深度学习的漏洞检测提供了新的技术思路和实证依据。

5.1　基于图神经网络的漏洞检测

5.1.1　实验设置

为系统评估基于图神经网络的漏洞检测方法性能，本研究从基础性能比较与核心组件验证两个维度展开实证分析。在基础性能评估方面，通过横向对比实验量化本模型与现有最优方法在准确率、召回率和 F1 值等核心指标的差异。针对模型核心组件的技术贡献，设计了三组实验验证其有效性，包括通过消融实验分析树结构语句表征机制对代码语义建模的影响，在相同实验环境下对比图注意力网络与其他 GNN 架构的性能差异，以及验证注意力池化相较常规方法的优势。实验设计旨在回答四个关键问题：

问题一：本研究提出的模型在漏洞检测性能上是否显著优于现有最优方法。

问题二：基于树结构的语句表征模块如何影响漏洞检测的性能。

问题三：不同的 GNN 模型如何影响漏洞检测的性能。

问题四：不同的图池化策略如何影响漏洞检测的性能。

5.1.2　数据集与基准模型

为评估 TreeGATTrans 模型的有效性，本研究采用 Devign、ReVeal 和 BigVul 三个基准漏洞检测数据集进行实验验证。这些数据集在样本规模、漏洞类型和数据构建方法等方面各具特点，能够全面检验模型的检测性能。以下具体说明各数据集的特征及其构建过程。

Devign[5]数据集来源于 4 个大型 C 语言开源项目（Qemu、Wireshark、FFmpeg 以及 Linux Kernel），包含约 12,000 个漏洞样本和 14,000 个非漏洞样本。在构建过程中，研究者首先分析代码提交信息筛选安全相关提交，随后区分漏洞修复提交（Vulnerability-

Fix Commits, VFCs）和非漏洞修复提交（Non-Vulnerability-Fix Commits，Non-VFCs），其中 VFCs 用于提取漏洞函数作为正样本，Non-VFCs 则提取非漏洞函数作为负样本。为确保数据质量，四名安全专家进行了两轮独立标注和交叉验证，累计投入 600 小时，使数据集具有较高的可靠性。

ReVeal[6]数据集基于 Linux Debian Kernel 和 Chromium 项目构建，包含约 2,000 个漏洞样本和 20,000 个非漏洞样本。该数据集旨在解决现有数据集中代码复杂度不足和漏洞分布偏离真实场景的问题，通过匹配真实项目的代码特征和漏洞类型分布，提高检测模型在实际环境中的适用性。研究者从 Chromium 和 Debian 安全跟踪器中提取已修复漏洞的公开补丁，筛选标记为安全相关的补丁文件，并收集补丁前后的 C/C++源码及头文件。在样本标注过程中，补丁中被修改函数的前置版本标记为漏洞样本，修复后的版本及未修改的函数均标记为非漏洞样本。这种基于真实漏洞修复记录的数据构建方法增强了数据场景的真实性。

BigVul[41]数据集由范等人从 348 个 C/C++开源 GitHub 项目中收集，覆盖 2002-2019 年 CVE 数据库记录的 91 种漏洞类型，包含约 10,000 个漏洞函数与 179,000 个非漏洞函数。研究者首先从 CVE 数据库爬取漏洞描述（含 CVE ID、严重性评分、CWE 分类等元数据）及关联的 GitHub 代码库链接，随后下载对应代码库并通过版本比对提取漏洞修复相关的代码变更。通过分析补丁前后的代码差异，该数据集成功还原了漏洞函数的易受攻击版本。该数据集完整保留了从 CVE 元数据到具体代码变更的溯源信息，支持细粒度的漏洞分析。

在数据预处理阶段，本研究使用 Joern 进行代码解析，剔除存在语法错误或超出 Joern 解析能力的异常函数样本。经清洗后的数据集统计信息详见表 1。

本研究选取了三种与 TreeGATTrans 参数量级相近的典型模型作为基准对比，包括基于序列的 SySeVR[3]、基于图的 Devign 与 ReVeal。SySeVR 通过程序切片技术提取代码依赖特征，利用控制依赖和数据依赖关系生成代码切片，并采用双向门控循环单元捕捉时序语义特征。Devign 构建了融合抽象语法树、控制流图、代码依赖图及自然代码结构的联合图表示，通过门控图神经网络实现上下文感知的漏洞检测。ReVeal 则采用代码属性图结合门控图神经网络提取语法和语义特征，并引入合成少数类过采样技术缓解数据不平衡问题，通过表示学习优化类别分离能力来提升检测性能。

表 1　漏洞检测数据集

数据集	样本总数	有漏洞的数量	无漏洞的数量	有漏洞的数量占比
FFMPeg+Qemu	26,533	1,2074	14,459	45.51%
ReVeal	18,632	1,711	16,921	9.18%
BigVul	18,3651	10,600	173,051	5.77%

5.1.3　性能评价指标

本实验选取 Precision、Recall、Accuracy、F1 作为评估指标。

Precision（精确率）：衡量模型预测结果的可靠性，表示被判定为有漏洞的样本中实际存在漏洞的比例：

$$Precision = \frac{TP}{TP + FP} \tag{5.1}$$

Recall（召回率）：衡量模型的漏洞检出能力，表示在实际存在的漏洞中被正确识别的样本比例：

$$Recall = \frac{TP}{TP + FN} \tag{5.2}$$

Accuracy（准确率）：衡量模型的整体预测准确度，表示所有样本中被正确分类的比例：

$$Accuracy = \frac{TP + TN}{P + N} \tag{5.3}$$

F1 Score：作为精确率和召回率的调和平均数，综合评估模型的平衡性能：

$$F1 = \frac{2TP}{FP + 2TP + FN} \tag{5.4}$$

其中，TP（True Positives）表示正确识别的漏洞样本数，TN（True Negatives）表示正确识别的非漏洞样本数，FP（False Positives）表示将非漏洞误判为漏洞的样本数，FN（False Negatives）表示漏检的真实漏洞样本数。这组指标从不同维度反映了模型在漏洞检测任务中的分类性能，能够客观评估模型的实用价值。

5.1.4　实验环境

本研究采用标准的实验环境配置，包括计算硬件、软件框架和训练参数等。实验平台基于配备 NVIDIA RTX 3090 GPU 的 Ubuntu 20.04.6 服务器搭建，采用 Python 3.11 作为主要开发环境，并集成 PyTorch 2.1.0 深度学习框架、DGL 2.2.1 图神经网络库和 Joern 2.0 静态代码分析工具。PyTorch 框架的动态计算图机制和 GPU 加速能力有效支持了模

型训练过程中的前向传播、反向传播和参数优化等关键计算任务。DGL 库通过优化的图数据结构和消息传递接口实现了高效的图神经网络运算，其 GPU 加速功能显著提升了大规模图计算的执行效率。Joern 工具则提供了静态代码分析能力，支持通过代码属性图对 C/C++等编程语言的漏洞模式进行深入分析。完整的环境配置参数见表 2。

实验采用统一的训练参数设置，批处理大小为 32，dropout 率为 0.2，优化器选用 Adam 并设定初始学习率为 2e-5（采用线性衰减策略），最大训练轮次为 30 epoch。所有隐藏层（含 TreeLSTM、GNN 和 MLP 等）维度统一设置为 512。为确保实验公平性，Devign 和 BigVul 数据集按 8:1:1 比例随机划分为训练集、验证集和测试集，ReVeal 数据集按 7:2:1 比例随机划分。所有划分过程均保持原始数据的类别分布不变。

表 2　实验环境配置

类别	配置项	版本/型号
硬件	GPU	NVIDIA RTX 3090
操作系统	OS	Ubuntu 20.04.6
静态分析工具	Joern	2.0
开发环境	Conda	Python 3.11.9
	深度学习框架	PyTorch 2.1.0
	图神经网络库	DGL 2.2.1

针对研究问题 1，实验选取基于序列模型的 SySeVR 与基于图模型的 Devign、ReVeal 作为对比基线。评估指标包括取精确率、召回率、准确率和 F1 值。

对于研究问题 2-4，实验保持与研究问题 1 相同的评估指标和数据集划分。研究问题 2 通过消融实验验证树结构语句表征模块的有效性；研究问题 3 对比图注意力网络、门控图神经网络与图卷积网络的检测性能差异；研究问题 4 评估注意力池化、求和池化、平均池化和最大池化策略在特征提取能力上的优劣。

5.1.5　实验结果与讨论

5.1.5.1 实验 1：本研究提出的模型在漏洞检测性能上是否显著优于现有最优方法

实验分析与讨论：表 3 所示的实验结果表明，本研究提出的 TreeGATTrans 模型在 Devign、ReVeal 和 BigVul 三个基准数据集上均超越基线模型 Devign、ReVeal 和 SySeVR，验证了基于树和图层次化学习框架的有效性，但其性能在不同数据集上呈现明显差异。

在 Devign 数据集上，TreeGATTrans 模型的 F1 值达到 63.96%，较基线模型实现了 1.77%~13.53%的绝对性能提升和 2.85%~26.83%的相对性能提升。在 ReVeal 数据集上，TreeGATTrans 模型的 F1 值达到了 42.25%，较基线模型实现了 0.63%~11.51%的绝对性能提升和 1.51%~37.44%的相对性能提升。这一结果得益于结构化简策略对关键语义路径的增强，通过将抽象语法树的原子节点归约至语句级节点，并保留控制依赖与数据依赖，有效抑制细粒度噪声对跨语句漏洞模式学习的干扰。同时，Tree-LSTM 对 AST 子树进行细粒度语句建模，图注意力网络学习语句节点之间的依赖关系，基于注意力机制的图池化方法聚合节点特征生成图级别表征，有效增强了模型对漏洞相关特征与依赖的表征建模能力。

在 BigVul 数据集上，TreeGATTrans 模型以 26.60%的 F1 值显著优于 SySeVR、Devign 和 ReVeal 等基线模型，后者分别获得 19.34%、20.98%和 22.87%的 F1 值。值得注意的是，基于 BERT 的编码器模型（具体见 5.2.5 节）在该数据集上与 TreeGATTrans 的性能差异显著，这一现象在其他基准数据集中并未出现。已有研究表明[66]，BigVul 数据集存在结构缺失、标签噪声和数据重复等固有缺陷，其中 17.6%的代码片段存在信息缺失，46.9%的漏洞标签因修复提交中混杂非实质性修改而产生偏差，17%的数据重复率进一步增加了模型过拟合的风险。

BigVul 数据集存在的代码信息缺失会破坏代码属性图的完整性，具体表现为控制依赖和数据依赖链的断裂以及孤立节点的产生。TreeGATTrans 模型依赖完整的图结构来实现层次化语义学习，而 AST 子树的断裂会影响 Tree-LSTM 的细粒度语法建模能力，全局依赖传播路径的偏差则会阻碍跨语句漏洞特征的准确提取。相比之下，基于 BERT 的编码器模型通过自注意力机制捕捉上下文语义，对全局结构完整性的要求较低，因此在代码缺失时表现出更好的适应性。此外，标签噪声与模型复杂度的相互作用进一步加剧了性能差异。作为参数量级较小的图模型，TreeGATTrans 的层次化特征学习机制更容易受到噪声标签的干扰。而 BERT 类的复杂模型可能通过拟合训练集中的错误标签，在测试集上表现出伪高准确率，但这种高准确率与标签纠偏后的真实性能可能存在差异。

综合来看，BigVul 数据集的数据集特性与图结构模型的基本假设之间的冲突，导致 TreeGATTrans 与编码器模型的性能差异被放大。其他数据集的实验结果证实，在数据质量较高的情况下，层次化图学习框架仍能展现出显著优势。

实验结论：实验表明，本研究提出的基于图神经网络的漏洞检测框架 TreeGATTrans 通过结构化简策略与基于树和图的层次化表征学习机制，在 Devign、ReVeal 与 BigVul 数据集上的 F1 值分别达到 63.96%、42.25%和 26.60%，相较于多个基线模型，绝对提升 0.63%~13.53%，相对提升 1.51%~37.54%。

表 3　TreeGATTrans 与基线模型在各漏洞检测数据集上的性能对比

Dataset	Model	Accuracy	Precision	Recall	F1
Devign	SySeVR	47.85	46.06	58.81	51.66
	Devign	58.12	55.27	46.37	50.43
	ReVeal	**61.07**	**55.50**	70.70	62.19
	TreeGATTrans（本文方法）	55.84	51.49	**84.42**	**63.96**
ReVeal	SySeVR	74.33	40.07	24.94	30.74
	Devign	87.49	31.55	36.65	33.91
	ReVeal	81.77	31.55	**61.14**	41.62
	TreeGATTrans（本文方法）	**92.36**	**44.12**	40.54	**42.25**
BigVul	SySeVR	90.10	**30.91**	14.08	19.34
	Devign	**92.78**	30.61	15.96	20.98
	ReVeal	87.14	17.22	**34.04**	22.87
	TreeGATTrans（本文方法）	91.69	25.79	27.47	**26.60**

5.1.5.2　实验 2：基于树结构的语句表征模块如何影响漏洞检测的性能

实验分析与讨论：为评估树结构建模对漏洞检测性能的贡献，本研究通过消融实验对比移除 Tree-LSTM 模块（简称 TreeGATTrans w/o tree）的模型性能差异。

如表 4 所示，引入基于抽象语法树结构的语句表征学习后，模型在 Devign、ReVeal 与 BigVul 数据集上的 F1 值分别实现 2.51%、11.95%与 5.41%的绝对提升，对应的相对提升幅度达 4.08%、39.44%与 25.53%。实验结果表明，树结构学习对模型性能的提升幅度在各数据集中存在差异，其中 ReVeal 数据集展现出最显著的性能增益。这种性能提升源于 Tree-LSTM 模块独特的语法结构建模机制。该模块通过 AST 拓扑关系自底向上递归聚合子节点状态，并利用门控单元实现基于语法层级的特征筛选，相较于扁平化序列建模方法，能更精确地捕获语句的语法语义特征。同时，该模块生成的语法感知向量

为后续的图神经网络模块提供了高质量的特征向量。当模型学习语句间粗粒度依赖关系时，图网络能够依据语句内部已学习的类型和操作符语义等细粒度特征，动态调整关系权重以优化上下文信息的聚合。

实验结论：实验结果表明，基于抽象语法树的语句表征学习方法通过有效捕捉程序语法关系和语义特征，显著提升了漏洞检测性能。在 Devign、ReVeal 和 BigVul 数据集上，TreeGATTrans 相比于 TreeGATTrans w/o tree，其 F1 值绝对提升幅度分别为 2.51%、11.95%和 5.41%，相对提升达到 4.08%、39.44%和 25.53%。该方法通过识别关键语法元素并有效抑制噪声干扰，显著改善了模型对程序语义的理解能力，从而提高了漏洞检测的可靠性。

表 4　TreeGATTrans 消融树表征学习模块对漏洞检测性能的影响

Dataset	Model	Accuracy	Precision	Recall	F1
Devign	TreeGATTrans	**55.84**	**51.49**	**84.42**	**63.96**
	TreeGATTrans wo tree	55.09	51.08	77.11	61.45
ReVeal	TreeGATTrans	**92.36**	**44.12**	**40.54**	**42.25**
	TreeGATTrans wo tree	91.43	34.48	27.03	30.30
BigVul	TreeGATTrans	91.69	25.79	**27.47**	**26.60**
	TreeGATTrans wo tree	**94.12**	**39.89**	14.43	21.19

5.1.5.3　实验 3：不同的 GNN 模型如何影响漏洞检测的性能

实验分析与讨论：为探究不同图神经网络架构对漏洞检测性能的影响，本研究对比了基于图注意力网络（GAT）、门控图神经网络（GGNN）和图卷积网络（GCN）的 TreeGATTrans 变体模型。如表 5 的实验数据表明，GAT 凭借其注意力机制在全局特征建模上的优势，在三类数据集中均展现出显著的性能提升，验证了其在捕捉代码语句语义关联中的有效性。

在 Devign 数据集中，GAT 模型以 63.96%的 F1 值领先于 GGNN（F1 值为 55.2%）和 GCN（F1 值为 58.75%），其绝对提升分别达 8.7%和 5.12%，对应相对提升为 15.74%和 8.87%。值得注意的是，GAT 的召回率高达 84.42%，远超 GGNN 与 GCN，表明注意力机制能更全面地覆盖漏洞相关节点。

在 ReVeal 数据集中，GAT 模型以 42.25%的 F1 值显著优于 GGNN（F1 值为 38.71%）和 GCN（F1 值为 28.57%），其相对提升分别达 9.14%和 47.88%。BigVul 数据集进一步验证了 GAT 的鲁棒性，在整体性能较低的场景下，GAT 的 F1 值仍较 GGNN（F1 值为 19.52%）和 GCN（F1 值为 20.18%）分别提升 36.27%和 31.81%，且召回率显著高于后两者。对比分析发现，传统图神经网络存在固有局限性，GCN 的固定卷积核导致特征融合过程中关键语义信息被稀释，而 GGNN 基于固定拓扑结构的消息传递范式，缺乏显式的注意力权重分配机制，导致其在代码属性图中难以自适应地区分关键漏洞模式节点与语法噪声节点

表 5　不同图神经网络的性能对比

Dataset	Model	Accuracy	Precision	Recall	F1
Devign	TreeGATTrans w GAT	55.84	51.49	**84.42**	**63.96**
	TreeGATTrans w GGNN	56.07	52.40	58.44	55.26
	TreeGATTrans w GCN	**57.57**	**53.54**	65.10	58.75
ReVeal	TreeGATTrans w GAT	92.36	44.12	**40.54**	**42.25**
	TreeGATTrans w GGNN	**92.92**	**48.00**	32.43	38.71
	TreeGATTrans w GCN	89.76	27.50	29.73	28.57
BigVul	TreeGATTrans w GAT	**91.69**	**25.79**	**27.47**	**26.60**
	TreeGATTrans w GGNN	88.74	16.05	24.90	19.52
	TreeGATTrans w GCN	89.54	17.35	24.11	20.18

实验结论：图神经网络的结构设计对漏洞检测性能具有关键影响。基于注意力机制的 GAT 模型通过动态权重分配策略，在 Devign、ReVeal 和 BigVul 数据集上分别较次优模型实现 15.74%、9.14%和 36.27%的 F1 值相对提升，验证了其在全局语义建模中的优越性。传统模型如 GGNN 和 GCN 因结构限制，难以有效捕捉代码图中的复杂依赖路径，导致关键特征丢失或噪声过度响应。实验结果表明，注意力机制通过加权特征聚焦能力，能够显著提升模型对漏洞模式的检测鲁棒性。

5.1.5.4　实验 4：不同的图池化策略如何影响漏洞检测的性能

实验分析与讨论：为探究图池化机制对漏洞检测性能的影响，本研究对比分析了注意力池化、求和池化、最大值池化及平均池化策略的检测性能。实验结果表明，基于注

意力机制的池化策略由于采用动态权重分配方式，在多个数据集上均表现出最优性能。如表 6 所示，该方法能够有效聚焦与漏洞相关的关键节点特征，缓解传统池化方法在全局特征聚合过程中存在的关键语义信息丢失问题。

表 6　不同池化策略的性能对比

Dataset	Model	Accuracy	Precision	Recall	F1
Devign	TreeGATTrans	55.84	51.49	**84.42**	**63.96**
	TreeGATTrans w sum poolling	56.14	52.22	64.94	57.89
	TreeGATTrans w max poolling	57.05	52.87	68.83	59.80
	TreeGATTrans w avg poolling	**59.91**	**55.80**	65.58	60.30
ReVeal	TreeGATTrans	92.36	44.12	**40.54**	**42.25**
	TreeGATTrans w sum poolling	89.57	26.83	29.73	28.21
	TreeGATTrans w max poolling	**93.67**	**63.64**	18.92	29.17
	TreeGATTrans w avg poolling	93.48	54.55	32.43	40.68
BigVul	TreeGATTrans	91.69	25.79	**27.47**	**26.60**
	TreeGATTrans w sum poolling	94.21	39.55	10.47	16.56
	TreeGATTrans w max poolling	**94.25**	**42.86**	14.82	22.03
	TreeGATTrans w avg poolling	94.06	38.71	14.23	20.81

在 Devign 数据集上，TreeGATTrans 模型采用注意力池化获得 84.42%的召回率和 63.96%的 F1 值，明显优于其他池化方法。这种性能提升主要得益于自注意力机制对高漏洞概率节点的动态加权能力，能够有效区分关键节点与噪声节点。相比之下，求和池化（F1 值为 57.89%）、平均池化（F1 值为 60.30%）和最大值池化（F1 值为 59.80%）由于采用均质化或极值依赖的聚合方式，在特征提取过程中均出现不同程度的语义信息丢失问题，进而削弱模型对复杂依赖路径的覆盖能力。

这一现象在 ReVeal 数据集中表现更为突出，注意力池化（F1 值为 42.25%）仍保持相对稳定的检测能力，而最大值池化（F1 值为 29.17%）因过度聚焦局部极值特征，致使召回率骤降至 18.92%，反映出静态池化策略在建模远距离依赖关系时的固有缺陷。BigVul 数据集进一步验证了注意力池化的鲁棒性，其 26.60%的 F1 值显著高于求和池化（F1 值为 16.56%）、最大值池化（F1 值为 22.03%）和平均池化（F1 值为 20.81%）。值得注意的是，静态池化策略在该数据集上的召回率普遍低于 14.82%，其中求和池化仅达

到 10.47%，表明这些方法在噪声干扰下的特征筛选能力存在明显不足。

总的来说，传统池化策略在漏洞检测任务中存在固有局限性。最大值池化由于仅保留单维度极值特征，往往导致重要语义信息丢失；求和池化对节点数量敏感，容易引入冗余噪声特征；平均池化则因特征均衡化处理而难以突出关键节点。相比之下，注意力池化通过多层自注意力机制动态学习全局节点重要性权重，在特征融合过程中有效克服了传统图神经网络因局部邻域聚合产生的结构性偏差，特别是缓解了过平滑效应问题，实现了程序语义特征的有效整合。这种机制能够根据节点特征自适应地调整聚合权重，从而更好地保留漏洞检测所需的关键语义信息。

实验结论：实验结果表明，特征聚合机制的选择对漏洞检测性能具有关键影响。基于注意力机制的动态权重聚合方法能够有效识别与漏洞相关的语义特征并抑制噪声干扰。相比之下，静态池化方法由于采用固定聚合策略，难以在精确率和召回率之间取得平衡，导致关键特征信息丢失或局部语义过拟合等问题。

5.2　图结构与预训练模型融合的漏洞检测

5.2.1　实验设置

为系统评估本研究提出的图结构与预训练模型融合框架在漏洞检测领域的有效性，本研究从基础性能比较、核心组件验证和泛化性验证三个维度展开实证分析。在基础性能评估层面，通过横向对比实验量化评估本模型与现有最优方法在准确率、召回率和 F1 值等核心指标的差异。针对模型核心组件的技术贡献，通过消融实验评估图结构融合策略对漏洞检测性能的提升幅度，并比较不同融合方式对模型性能的影响程度。在方法泛化性验证方面，主要评估 GNNAdapter 在不同预训练基座上的泛化能力。基于此，本研究重点探讨以下四个核心问题：

问题一：GNNAdapter 相较于基线方法在漏洞检测任务中的性能优势。

问题二：图结构融合策略对模型性能提升的具体贡献度。

问题三：不同图神经网络与预训练模型融合方式的性能差异。

问题四：GNNAdapter 是否能泛化到不同预训练基座模型。

5.2.2　数据集与基准模型

为验证 GNNAdapter 的有效性，本研究选取 Devign、ReVeal 与 BigVul 三个主流漏

洞检测数据集开展实验，关于数据集的具体介绍详见 5.1.2 节。

在评估实验中，本研究选用两个最先进的基于图的方法（Devign 和 ReVeal，相关介绍详见 5.1.2 节）、三个基于序列的方法（LineVul、PILOT 和 StagedVulBERT）和一个基于图和序列的方法（ReGVD）进行比较。

LineVul[74]将源代码视为一段自然语言序列，采用字节对编码将源代码分割为词元序列，借助 CodeBERT 预训练模型生成上下文感知的词元嵌入向量，通过 Transformer 的自注意力机制捕获代码词元间的远距离依赖关系，并以[CLS]标记的最终层向量作为程序全局表征进行漏洞判定。

PILOT[59]采用半监督学习框架进行漏洞检测，首先基于已标注漏洞代码的语义特征，通过最大距离差异原则将未标注代码片段初始化为非漏洞伪标注样本；然后使用 CodeBERT 预训练模型，在真实漏洞样本与伪标注非漏洞样本的混合数据集上训练初始检测模型；最后通过置信度筛选机制，选取高分类概率的未标注样本作为新增伪标注数据，迭代式扩充训练集并对模型进行微调优化，以缓解代码漏洞检测任务中高质量标注数据稀缺的问题。

StagedVulBERT[69]是一种基于 CodeBERT 的多阶段漏洞检测框架，通过分层语义建模与长序列优化机制提升检测性能。其采用 TETransformer 和 SETransformer 分别捕获代码的局部语法特征与全局上下文依赖，并设计掩码语句预测任务增强语义理解。针对长代码输入，框架通过滑动窗口分割与门控融合算法保留完整语义信息，同时利用粗粒度和细粒度标签联合训练实现多级漏洞定位。在本研究的对比实验中，为保持评估一致性，仅采用其粗粒度检测结果作为基线参照。

ReGVD[75]通过将原始源代码视为一个扁平的标记序列来构建图，利用预训练模型的词元嵌入层初始化节点特征，然后使用图神经网络、图级别求和与最大池化来学习源代码的图表征，以实现代码漏洞检测。

5.2.3 性能评价指标

本实验与基于图神经网络的漏洞检测一致，采用精确率、召回率、准确率和 F1 值作为评估指标，以全面衡量模型的检测性能。

5.2.4 实验环境

实验采用统一的硬件配置和超参数设置以确保结果可比性。具体而言，批处理大小

设置为 32 或 64，Dropout 率为 0.3，优化器选用 Adam 并设定初始学习率为 2e-5（采用线性衰减策略），最大训练轮次限制为 30 轮。所有隐藏层（包括 TreeLSTM、GNN 和 MLP）维度统一设置为 512。为保障实验公平性，Devign 和 BigVul 数据集按 8:1:1 比例随机划分为训练集、验证集和测试集，ReVeal 数据集按 7:2:1 比例随机划分。所有划分过程均保持原始数据的类别分布不变。

针对研究问题 1，实验选取三类基线模型进行对比：基于图的模型（Devign、ReVeal）、基于序列的模型（LineVul、PILOT 和 StagedVulBERT）及基于图和序列的模型（ReGVD），评估采用精确度、召回率、准确率和 F1 值四项核心指标，其中除 ReVeal 因原 Google Cloud 服务不可用而直接引用文献复现数据外，其余模型均基于开源代码实现，并严格遵循原论文的超参数设置以确保公平比较。需说明的是，StagedVulBERT 未公开其预训练阶段代码，故直接采用其发布的预训练检查点进行初始化。

在研究问题 2 的实验中，沿用 RQ1 的数据集划分方式及评估指标，通过控制变量消融实验验证 GNNAdapter 架构中图结构编码器的有效性。研究问题 3 探究不同融合方式对模型性能的影响，通过端到端训练比较 GNN 与预训练模型的多种融合策略，分析其对漏洞检测效果的差异。最后，研究问题 4 基于相同的实验基准，将 GNN 模块分别与编码器架构模型（CodeBERT）和解码器架构模型（Phi-1）进行组合实验，以验证框架在不同预训练模型上的泛化能力。

5.2.5　实验结果与讨论

5.2.5.1　实验 1: GNNAdapter 相较于基线方法在漏洞检测任务中的性能优势

实验分析与讨论：本研究通过对比 GNNAdapter 模型与当前最先进的漏洞检测方法，验证了图结构与预训练模型融合框架在漏洞检测任务中的有效性，如表 7 所示的实验结果表明，在 Devign、ReVeal 和 BigVul 三个数据集上，GNNAdapter 展现出显著优势，下面从不同模型对比角度展开分析。

相较于传统图神经网络模型，GNNAdapter 在 Devign 数据集上的 F1 值达到 67.26%，较 Devign 和 ReVeal 模型基线分别取得 9.31%和 5.07%的绝对提升，对应相对提升为 16.07%和 8.15%，验证了图结构融合机制对代码特征表征能力的增强作用。该优势在 ReVeal 数据集上进一步扩大，F1 值分别绝对提升 20.96%和 13.25%，相对提升 61.81%和 31.84%。而 BigVul 数据集中的性能差距更为突出，其 F1 值较基线模型平均绝对提升

71.8%，反映出传统图模型在非完整代码片段表征方面的局限性。通过融合图结构与预训练模型，GNNAdapter 在该数据集上获得 93.72%的 F1 值，有效弥补了传统方法的特征表征缺陷。

表 7　GNNAdapter 与基线模型在各漏洞检测数据集中的性能对比

Dataset	Model	Accuracy	Precision	Recall	F1
Devign	Devign	56.89	52.50	64.67	57.95
	ReVeal	61.07	55.50	70.70	62.19
	ReGVD	62.96	**60.65**	55.14	57.76
	LineVul	63.79	60.60	58.44	59.50
	PILOT	56.18	52.23	54.02	53.11
	StagedVulBERT	63.75	57.76	75.83	65.57
	GNNAdapter（本文方法）	**64.05**	58.26	**79.55**	**67.26**
ReVeal	Devign	87.49	31.55	36.65	33.91
	ReVeal	81.77	31.55	61.14	41.62
	ReGVD	90.48	55.81	16.00	24.87
	LineVul	**92.04**	**65.14**	41.11	50.41
	PILOT	86.06	37.46	44.67	40.75
	StagedVulBERT	89.21	45.30	46.00	45.64
	GNNAdapter（本文方法）	90.50	40.79	**83.78**	**54.87**
BigVul	Devign	92.78	30.61	15.96	20.98
	ReVeal	87.14	17.22	34.04	22.87
	ReGVD	98.22	94.33	72.51	81.99
	LineVul	98.89	95.87	83.70	89.37
	PILOT	83.81	17.15	49.48	25.48
	StagedVulBERT	99.28	**96.18**	90.81	93.42
	GNNAdapter（本文方法）	**99.33**	96.06	**91.50**	**93.72**

相较于序列模型，GNNAdapter 同样展现出显著优势。在 Devign 数据集上，相比于其他三个序列模型 LineVul（F1 值为 59.50%）、PILOT（F1 值为 53.11%）和 StagedVul-BERT（F1 值为 65.57%），GNNAdapter 的 F1 值实现 7.87%的平均绝对提升。其中性能

最优的模型 StagedVulBERT 较 GNNAdapter 仍存在 1.69%的性能差距。在召回率指标上，LineVul 和 PILOT 较 GNNAdapter 分别下降 21.11%和 25.53%，这源于纯序列模型对跨语句依赖关系的建模缺陷。尽管 StagedVulBERT 通过语句分割和 Transformer 编码部分缓解了该问题，但其简单分割策略可能会导致冗余信息引入和跨语句片段丢失，最终限制其性能提升空间。在 ReVeal 和 BigVul 数据集上，GNNAdapter 相较序列模型分别取得平均 9.27%和 24.3%的 F1 值绝对提升。尽管 LineVul 和 StagedVulBERT 表现较优，但仍落后 GNNAdapter 0.3~4.35%。

相较于基于图和序列的模型 ReGVD，GNNAdapter 通过构建代码属性图策略展现出显著优势。在 Devign、ReVeal 和 BigVul 三个基准数据集上，GNNAdapter 的 F1 值相较于 ReGVD 分别实现 9.5%、30%和 11.73%的绝对提升，对应的相对提升为 16.45%、120.63%和 14.31%。ReGVD 将源代码线性词元序列直接映射为图节点，这导致节点特征仅编码词汇级语义（由预训练嵌入初始化），而缺失对抽象语法树、控制流等结构化信息的建模能力。这种构建策略产生的图结构本质上仍是序列化表征的拓扑展开，未能有效建立跨语句的语义依赖，致使节点间高阶交互受限，无法充分建模漏洞相关的语法语义。值得注意的是，ReGVD 在 BigVul 数据集（F1 值为 81.99%）的较好表现揭示其词元级图结构在局部检测中的有效性，但与本方法仍然存在较大差距。

实验结论：本研究提出的 GNNAdapter 模型在漏洞检测任务中展现出显著优势。实验表明，通过代码结构图（GNN）与语义嵌入（CodeT5+）的特征融合机制，模型在 Devign、ReVeal 和 BigVul 数据集上均实现了最优的 F1 值，相较于基线绝对提升 0.3%~72.74%，相对提升 0.32%~346.71%。通过融合策略，模型有效捕捉了传统方法难以识别的跨语句漏洞模式，使检测性能显著提升。

5.2.5.2　实验 2：图结构融合策略对模型性能提升的具体贡献度

实验分析与讨论：实验通过对比 CodeT5+编码器在引入 GNNAdapter 前后的性能差异，验证了代码结构信息在漏洞检测中的关键作用。如表 8 所示的实验表明，结构和语义融合机制在不同数据分布场景下展现出不同的性能增益。

在类别均衡的 Devign 数据集（正样本占比 45.51%）中，相比于不使用图结构融合策略的预训练模型 CodeT5+编码器，GNNAdapter 将召回率从 72.27%提升至 79.55%，F1 值由 64.4%增长至 67.26%，证实了代码结构特征对语义特征的补充可有效增强潜在

漏洞模式的捕获能力。在极端类别不平衡的 ReVeal 数据集（正样本占比 9.18%）上，结构信息的价值更为显著。GNNAdapter 使召回率从 38.89%提升至 83.78%，F1 值由 46.11%提升至 54.87%。这表明，通过建模控制依赖、数据依赖等结构关系，能有效增强模型对漏洞样本的捕获。在高基线水平的 BigVul 数据集（正样本占比 5.77%）中，GNNAdapter 将 F1 值从 92.68%提升至 93.72%，将精确率从 94.84%提升至 96.06%，将召回率从 90.62%提升至 91.5%。

实验结论：本研究提出的 GNNAdapter 通过将代码结构图特征与代码语义特征深度融合，显著提升了漏洞检测模型的性能。相较于不使用图结构信息的基线模型，GNNAdapter 在 Devign、ReVeal 和 BigVul 数据集上均实现了最优的 F1 值和召回率，其中 F1 分别提升 2.86%、8.76%与 1.04%，召回率分别提升 7.28%、44.89%与 0.88%。

表 8　GNNAdapter 消融图模块对漏洞检测性能的影响

Dataset	Model	Accuracy	Precision	Recall	F1 Score
Devign	GNNAdapter wo gnn	63.64	**62.70**	72.27	64.40
	GNNAdapter	**64.05**	58.26	**79.55**	**67.26**
ReVeal	GNNAdapter wo gnn	**91.05**	**56.63**	38.89	46.11
	GNNAdapter	90.50	40.79	**83.78**	**54.87**
BigVul	GNNAdapter wo gnn	99.20	94.84	90.62	92.68
	GNNAdapter	**99.33**	**96.06**	**91.50**	**93.72**

5.2.5.3　实验 3：不同图神经网络与预训练模型融合方式的性能差异

实验分析与讨论：本节针对图神经网络与预训练模型的融合策略展开性能对比分析。如表 9 所示的实验结果表明，在 Devign、ReVeal 和 BigVul 三个漏洞检测基准数据集上，图结构引导的深度注意力融合（GAF）相较浅层拼接融合（GCF）展现出较大优势。具体而言，GAF 在 Devign 数据集实现 67.26%的 F1 值，较 GCF 提升 3.19%；在 ReVeal 数据集达到 54.87%的 F1 值，提升幅度达 7.84%。

从特征融合机制分析，GCF 采用浅层静态拼接方式，虽能有效保留图结构信息，但难以将结构特征与词元语义特征进行深度交互与融合。相较而言，GAF 通过将图结构表征作为软提示注入 Transformer 的注意力计算层，实现了词元级语义权重的动态调整，以更准确地捕获与漏洞相关的语义特征。值得注意的是，在 BigVul 数据集上， GAF

（93.72%）与 GCF（93.59%）呈现微小性能差异（0.13%），这表明当漏洞模式主要依赖局部语义特征时，浅层融合即可提供简便解决方案。

从模型的预训练任务角度分析，CodeT5+是一个编码器-解码器架构，在预训练过程中，编码器不仅学习语义表征，还根据前缀提示生成适配解码器的中间表征。这种动态调整输入结构的能力与 GAF 天然适配。因此，当图结构信息通过软提示方式（GAF）注入到 CodeT5+编码器时，其可有效协调词元序列与软提示的交互与学习，从而提升模型对代码语法语义的建模能力。

表 9　预训练模型 CodeT5+与图结构的不同融合策略性能对比

Dataset	Model	Accuracy	Precision	Recall	F1 Score
Devign	GAF	**64.05**	**58.26**	**79.55**	**67.26**
	GCF	60.78	54.94	76.82	64.07
ReVeal	GAF	**90.50**	**40.79**	**83.78**	**54.87**
	GCF	89.57	37.66	78.38	50.88
BigVul	GAF	**99.33**	96.06	**91.50**	**93.72**
	GCF	99.23	**96.10**	91.20	93.59

实验结论：GNN 与 CodeT5+的融合策略对漏洞检测性能具有较大影响，深度注意力融合（GAF）通过结构特征与语义特征的动态交互，在多数场景下性能更优，在 Devign、ReVeal 和 BigVul 数据集上分别较浅层拼接融合（GCF）实现 3%、4%和 0.13%的 F1 值绝对提升，以及 4.98%、7.84%和 0.14%的 F1 值相对提升。

5.2.5.4　实验 4：GNNAdapter 是否能泛化到不同预训练基座模型

实验分析与讨论：为验证本研究提出的图结构与预训练模型融合框架的泛化性，实验选取 CodeBERT 与 Phi-1 作为典型预训练基座模型，并系统评估引入 GNNAdapter 模块后的性能差异。如表 10 所示，该方法在不同模型架构和数据集条件下均表现稳定，其融合策略的效果与模型架构特性密切相关。为便于表述，后续分析中将图结构感知的浅层拼接融合和深度注意力融合分别简称为 GCF 与 GAF。

在编码器模型 CodeBERT 的实验中，两种融合策略均显著提升了漏洞检测性能，验证了结构特征融合对增强漏洞语法特征识别的有效性。实验数据显示 GCF 的检测性能整体优于 GAF。在 Devign 数据集上，GCF 将 F1 值从 59.50%提升至 65.35%，相对提升

9.83%，而 GAF 仅提升至 64.60%，相对提升 8.57%。在 ReVeal 数据集上，GCF 的 F1 值从 50.41%提升至 56.57%，相对提升 12.22%，这一提升幅度显著高于 GAF 的 4.21%相对提升。这种性能差异可能与 CodeBERT 的预训练机制相关，其基于掩码语言建模和替换词元检测任务的训练方式，虽擅长捕捉局部词元共现关系，但对结构化特征的动态感知能力存在局限。值得注意的是，GAF 中图嵌入与词元嵌入的特征空间差异可能阻碍特征分布对齐，而绝对位置编码机制在插入前缀时引发的全局位置偏移问题，也可能破坏预训练阶段建立的位置分布模式。相比之下，GCF 采用静态拼接方法，将[CLS]标记的全局语义信息与图结构表征进行解耦式融合，实现了结构知识的注入。

表 10　不同预训练模型（codebert/phi-1）与图结构融合的漏洞检测性能对比

Dataset	Model	Fusion method	Accuracy	Precision	Recall	F1
Devign	codebert	none	**63.79**	**60.60**	58.44	59.50
		GAF	58.21	52.57	**83.77**	64.60
		GCF	61.53	55.38	79.72	**65.35**
	phi-1	none	61.74	58.19	56.11	57.13
		GAF	62.29	59.34	67.92	63.34
		GCF	**63.80**	**60.00**	**73.58**	**66.10**
ReVeal	codebert	none	**92.04**	**65.14**	41.11	50.41
		GAF	91.25	41.94	70.27	52.53
		GCF	91.99	45.16	**75.68**	**56.57**
	phi-1	none	89.89	45.45	13.33	20.62
		GAF	92.54	42.86	**33.33**	37.50
		GCF	**94.78**	**83.33**	27.78	**41.67**

在解码器架构的 Phi-1 模型中，两种融合策略同样展现出显著性能提升，验证了显式结构特征融合对自回归模型全局结构建模能力的增强作用。实验数据显示，GCF 在 Phi-1 模型上表现更为突出。在 Devign 数据集上，GCF 将 F1 值从 57.13%提升至 66.10%，相对提升 15.70%，而 GAF 仅提升至 63.34%，相对提升 10.87%。在 ReVeal 数据集上，GCF 的 F1 值从 20.62%大幅提升至 41.67%，相对提升 102.18%，这一提升幅度明显高于 GAF 的 81.86%相对提升。这些结果表明，GCF 更适配 Phi-1 的解码器架构特性。值得注意的是，Phi-1 采用的 BitFit 轻量化微调策略（仅更新偏置参数）可能对融合效果产生

影响。参数冻结机制限制了模型对注意力权重矩阵的调整能力，导致与异构图表征的动态交互难以有效实现。在此约束条件下，GCF 通过直接整合图嵌入与末位隐藏状态（[GL]标记），规避了复杂的注意力交互机制，为分类层提供了更有效的融合特征。

实验结论：实验结果表明，GNNAdapter 具备跨模型架构的泛化能力，其性能提升幅度取决于融合策略与模型架构特性。具体而言，编码器模型 CodeBERT 在 Devign 和 ReVeal 数据集通过拼接融合分别取得平均 5.75%和 4.14%的 F1 值提升；解码器模型 Phi-1 则分别获得 7.59%和 18.97%的显著性能增益。这些发现证实，基于图结构的适配器模块能够通过特征空间融合，有效增强不同预训练范式下模型对程序语义的结构化建模能力，以提升在下游漏洞检测任务中的性能表现。

5.3 本章小节

为系统评估本研究提出的基于图神经网络的漏洞检测框架，以及图结构与预训练模型融合框架在漏洞检测任务上的性能，本章主要从整体性能评估与核心机制验证两个维度展开实验。具体而言，首先详细介绍了研究问题，数据集与基准模型，实验环境与设置；然后对实验结果进行了分析与讨论。经实验验证，本研究提出的层次化学习框架通过结构化简策略与基于树与图的层次化表征学习机制，在 Devign、ReVeal 与 BigVul 数据集上的 F1 值分别达到 63.96%、42.25%和 26.60%，均优于传统方法的性能。此外，本研究提出的 GNNAdapter 模型通过融合代码结构特征与文本语义特征，在 Devign、ReVeal 和 BigVul 数据集上的 F1 值分别达到了 67.26%、54.87%和 93.72%，均超过了所有基准模型。

总结与展望

总结：

当前软件系统的规模扩展与复杂度提升显著增加了漏洞发生概率，给信息安全带来严峻挑战。现有漏洞检测方法主要依赖图神经网络或序列模型提取代码特征，并通过分类器实现漏洞检测。然而这些方法在特征表征方面存在局限性。

传统基于图神经网络的漏洞检测方法受限于代码属性图中中间变量声明和非关键控制分支等冗余节点的干扰，导致关键节点间语义关联被噪声弱化；同时，其消息传递机制通常依赖局部邻居聚合，层数过少时无法捕捉远距离依赖，层数过多则可能易引发过平滑问题，导致所有节点的特征趋同。另一方面，以词元粒度进行预训练的序列模型对结构化语义（如控制依赖和数据依赖）的表征能力不足，导致隐含漏洞的关键特征学习不充分。针对上述问题，本研究提出两套创新性解决方案。

为解决传统图神经网络在噪声抑制、建模远距离依赖的局限性，本研究提出基于图神经网络的层次化学习架构。该架构通过结构化简策略将抽象语法树原子节点（如标识符和表达式等）归约至其所属语句节点，在保留控制依赖与数据依赖等关键路径的同时，有效抑制细粒度噪声。通过 Tree-LSTM 对语句节点的 AST 子树进行语法表征建模，以捕捉代码的底层语法结构；在此基础上，利用图注意力网络分析语句间的程序依赖关系，增强模型的上下文感知能力；最终，采用基于注意力机制的全局池化方法对节点特征进行重要性加权聚合，在保留关键漏洞敏感特征的同时，有效建模节点间的远距离依赖关系，从而生成更具判别力的图表征。

针对序列模型捕获控制依赖、数据依赖等结构化语义不足的问题，本研究构建图结构与预训练模型的融合框架。该框架包含两种特征融合范式：图结构感知的浅层拼接融合（GCF）通过特征向量解耦拼接，实现结构表征与文本表征的静态融合；图结构引导的深度注意力融合（GAF）将图结构表征作为软提示注入 Transformer 的注意力计算层，实现了词元级语义权重的动态调整，从而增强模型对程序结构化特征的建模能力。

实验结果表明，本研究所提出的基于图神经网络的漏洞检测框架在 Devign、ReVeal 与 BigVul 数据集上分别取得 63.96%、42.25%和 26.60%的 F1 值，显著优于传统基准模型。与此同时，图结构与预训练模型的融合框架在相同数据集上的 F1 值进一步提升至 67.26%、54.87%和 93.72%，全面优于现有最佳基线。

展望：

在基于图注意力的漏洞检测方法中，现有注意力机制聚焦于节点关联权重建模，但未能显式区分控制依赖、数据依赖等异质边的语义差异。针对该局限，未来研究可引入边类型感知的异质图神经网络，通过语义敏感的注意力权重分配机制增强代码结构表征能力。

针对图结构与预训练模型融合框架，当前图结构引导的深度注意力融合方法将图结构压缩为单一向量可能导致信息损失，后续可探索多向量压缩机制以保留更完整的结构信息。同时，受限于计算资源，当前研究尚未充分挖掘解码器架构大语言模型的适配潜力，亟待探索框架在千亿参数模型上的扩展性及其对漏洞语义的深层推理能力。进一步地，图结构前缀如何影响 Transformer 各层的权重分布与特征演化仍是黑箱问题，缺乏理论解释。需通过隐层特征可视化与权重归因分析揭示其作用机理，为模型提供可解释性支持。

参考文献

[1] Russell R, Kim L, Hamilton L, et al. Automated vulnerability detection in source code using deep representation learning[C]//2018 17th IEEE international conference on machine learning and applications (ICMLA). IEEE, 2018: 757-762.

[2] Li Z, Zou D, Xu S, et al. Vuldeepecker: A deep learning-based system for vulnerability detection[J]. arXiv preprint arXiv:1801.01681, 2018.

[3] Li Z, Zou D, Xu S, et al. Sysevr: A framework for using deep learning to detect software vulnerabilities[J]. IEEE Transactions on Dependable and Secure Computing, 2021, 19(4): 2244-2258.

[4] Cheng X, Zhang G, Wang H, et al. Path-sensitive code embedding via contrastive learning for software vulnerability detection[C]//Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis. 2022: 519-531.

[5] Zhou Y, Liu S, Siow J, et al. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks[J]. Advances in neural information processing systems, 2019, 32.

[6] Chakraborty S, Krishna R, Ding Y, et al. Deep learning based vulnerability detection: Are we there yet[J]. IEEE Transactions on Software Engineering, 2021.

[7] Wu Y, Zou D, Dou S, et al. VulCNN: An image-inspired scalable vulnerability detection system[C]//Proceedings of the 44th International Conference on Software Engineering. 2022: 2365-2376.

[8] Tang W, Tang M, Ban M, et al. CSGVD: A deep learning approach combining sequence and graph embedding for source code vulnerability detection[J]. Journal of Systems and Software, 2023, 199: 111623.

[9] Wen X C, Chen Y, Gao C, et al. Vulnerability detection with graph simplification and enhanced graph representation learning[C]//2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023: 2275-2286.

[10] Wang W, Nguyen T N, Wang S, et al. DeepVD: Toward Class-Separation Features for Neural Network Vulnerability Detection[C]//2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023: 2249-2261.

[11] Fu M, Tantithamthavorn C. Linevul: A transformer-based line-level vulnerability prediction[C]//Proceedings of the 19th International Conference on Mining Software Repositories. 2022: 608-620.

[12] Zhang J, Liu Z, Hu X, et al. Vulnerability Detection by Learning from Syntax-Based Execution Paths of Code[J]. IEEE Transactions on Software Engineering, 2023.

[13] Harzevili N S, Belle A B, Wang J, et al. A Survey on Automated Software Vulnerability Detection Using Machine Learning and Deep Learning[J]. arXiv preprint arXiv:2306.11673, 2023.

[14] Shin Y, Williams L. An initial study on the use of execution complexity metrics as indicators of software vulnerabilities[C]//Proceedings of the 7th International workshop on software engineering for secure systems. 2011: 1-7.

[15] Shar L K, Tan H B K. Predicting SQL injection and cross site scripting vulnerabilities through mining

input sanitization patterns[J]. Information and Software Technology, 2013, 55(10): 1767-1780.

[16] Lu G, Ju X, Chen X, et al. GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning[J]. Journal of Systems and Software, 2024, 212: 112031.

[17] Perl H, Dechand S, Smith M, et al. Vccfinder: Finding potential vulnerabilities in open-source projects to assist code audits[C]//Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security. 2015: 426-437.

[18] Li Z, Zou D, Xu S, et al. Vulpecker: an automated vulnerability detection system based on code similarity analysis[C]//Proceedings of the 32nd annual conference on computer security applications. 2016: 201-213.

[19] Kronjee J, Hommersom A, Vranken H. Discovering software vulnerabilities using data-flow analysis and machine learning[C]//Proceedings of the 13th international conference on availability, reliability and security. 2018: 1-10.

[20] Mikolov T, Chen K, Corrado G, et al. Efficient estimation of word representations in vector space[J]. arXiv preprint arXiv:1301.3781, 2013.

[21] Cho K, Van Merriënboer B, Gulcehre C, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation[J]. arXiv preprint arXiv:1406.1078, 2014.

[22] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017, 30.

[23] Zhang Z, Chen C, Liu B, et al. A Survey on Language Models for Code[J]. arXiv preprint arXiv:2311.07989, 2023.

[24] Li Y, Tarlow D, Brockschmidt M, et al. Gated graph sequence neural networks[J]. arXiv preprint arXiv:1511.05493, 2015.

[25] Chawla N V, Bowyer K W, Hall L O, et al. SMOTE: synthetic minority over-sampling technique[J]. Journal of artificial intelligence research, 2002, 16: 321-357.

[26] Li Y, Wang S, Nguyen T N. Vulnerability detection with fine-grained interpretations[C]//Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2021: 292-303.

[27] Ying Z, Bourgeois D, You J, et al. Gnnexplainer: Generating explanations for graph neural networks[J]. Advances in neural information processing systems, 2019, 32.

[28] Allen M, Horwitz S. Slicing Java programs that throw and catch exceptions[J]. ACM SIGPLAN Notices, 2003, 38(10): 44-54.

[29] Ferrante J, Ottenstein K J, Warren J D. The program dependence graph and its use in optimization[J]. ACM Transactions on Programming Languages and Systems (TOPLAS), 1987, 9(3): 319-349.

[30] Dey R, Salem F M. Gate-variants of gated recurrent unit (GRU) neural networks[C]//2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS). IEEE, 2017: 1597-1600.

[31] Tai K S, Socher R, Manning C D. Improved semantic representations from tree-structured long short-term memory networks[J]. arXiv preprint arXiv:1503.00075, 2015.

[32] Bellei C, Alattas H, Kaaniche N. Label-gcn: An effective method for adding label propagation to graph convolutional networks[J]. arXiv preprint arXiv:2104.02153, 2021.

[33] Pagliardini M, Gupta P, Jaggi M. Unsupervised learning of sentence embeddings using compositional

n-gram features[J]. arXiv preprint arXiv:1703.02507, 2017.

[34] Yamaguchi F, Golde N, Arp D, et al. Modeling and discovering vulnerabilities with code property graphs[C]//2014 IEEE Symposium on Security and Privacy. IEEE, 2014: 590-604.

[35] Feng Z, Guo D, Tang D, et al. Codebert: A pre-trained model for programming and natural languages[J]. arXiv preprint arXiv:2002.08155, 2020.

[36] Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[J]. arXiv preprint arXiv:1810.04805, 2018.

[37] Husain H, Wu H H, Gazit T, et al. Codesearchnet challenge: Evaluating the state of semantic code search[J]. arXiv preprint arXiv:1909.09436, 2019.

[38] Chen D, O'Bray L, Borgwardt K. Structure-aware transformer for graph representation learning[C]//International Conference on Machine Learning. PMLR, 2022: 3469-3489.

[39] Zhang J, Wang X, Zhang H, et al. A novel neural source code representation based on abstract syntax tree[C]//2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). IEEE, 2019: 783-794.

[40] Cao S, Sun X, Bo L, et al. Learning to Detect Memory-Related Vulnerabilities[J]. ACM Transactions on Software Engineering and Methodology, 2023.

[41] Fan J, Li Y, Wang S, et al. A C/C++ code vulnerability dataset with code changes and CVE summaries[C]//Proceedings of the 17th International Conference on Mining Software Repositories. 2020: 508-512.

[42] Liang H, Sun L, Wang M, et al. Deep learning with customized abstract syntax tree for bug localization[J]. IEEE Access, 2019, 7: 116309-116320.

[43] Ferrante J, Ottenstein K J, Warren J D. The program dependence graph and its use in optimization[C]//International Symposium on Programming. Berlin, Heidelberg: Springer Berlin Heidelberg, 1984: 125-132.

[44] Yamaguchi F, Golde N, Arp D, et al. Modeling and discovering vulnerabilities with code property graphs[C]//2014 IEEE symposium on security and privacy. IEEE, 2014: 590-604.

[45] Liu Z, Tang Z, Zhang J, et al. Pre-training by Predicting Program Dependencies for Vulnerability Analysis Tasks[C]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-13.

[46] Hu Y, Zhang Z, Zhao L. Beyond text: A deep dive into large language models' ability on understanding graph data[J]. arXiv preprint arXiv:2310.04944, 2023.

[47] Wang H, Feng S, He T, et al. Can language models solve graph problems in natural language?[J]. Advances in Neural Information Processing Systems, 2023, 36: 30840-30861.

[48] Guo J, Du L, Liu H, et al. Gpt4graph: Can large language models understand graph structured data? an empirical evaluation and benchmarking[J]. arXiv preprint arXiv:2305.15066, 2023.

[49] Wang J, Wu J, Hou Y, et al. Instructgraph: Boosting large language models via graph-centric instruction tuning and preference alignment[J]. arXiv preprint arXiv:2402.08785, 2024.

[50] Ye R, Zhang C, Wang R, et al. Language is all a graph needs[J]. arXiv preprint arXiv:2308.07134, 2023.

[51] Chai Z, Zhang T, Wu L, et al. Graphllm: Boosting graph reasoning ability of large language model[J]. arXiv preprint arXiv:2310.05845, 2023.

[52] Tian Y, Song H, Wang Z, et al. Graph neural prompting with large language models[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2024, 38(17): 19080-19088.

[53] Zhang X, Bosselut A, Yasunaga M, et al. Greaselm: Graph reasoning enhanced language models for question answering[J]. arXiv preprint arXiv:2201.08860, 2022.

[54] Zhu Y, Wang Y, Shi H, et al. Efficient tuning and inference for large language models on textual graphs[J]. arXiv preprint arXiv:2401.15569, 2024.

[55] Tang J, Qu M, Wang M, et al. Line: Large-scale information network embedding[C]//Proceedings of the 24th international conference on world wide web. 2015: 1067-1077.

[56] Perozzi B, Al-Rfou R, Skiena S. Deepwalk: Online learning of social representations[C]//Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining. 2014: 701-710.

[57] Grover A, Leskovec J. node2vec: Scalable feature learning for networks[C]//Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining. 2016: 855-864.

[58] Ren X, Tang J, Yin D, et al. A survey of large language models for graphs[C]//Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2024: 6616-6626.

[59] Wen X C, Wang X, Gao C, et al. When less is enough: Positive and unlabeled learning model for vulnerability detection[C]//2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, 2023: 345-357.

[60] Lin T Y, Goyal P, Girshick R, et al. Focal loss for dense object detection[C]//Proceedings of the IEEE international conference on computer vision. 2017: 2980-2988.

[61] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks[J]. arXiv preprint arXiv:1609.02907, 2016.

[62] Li Y, Tarlow D, Brockschmidt M, et al. Gated graph sequence neural networks[J]. arXiv preprint arXiv:1511.05493, 2015.

[63] Veličković P, Cucurull G, Casanova A, et al. Graph attention networks[J]. arXiv preprint arXiv:1710.10903, 2017.

[64] Clark K, Luong M T, Le Q V, et al. Electra: Pre-training text encoders as discriminators rather than generators[J]. arXiv preprint arXiv:2003.10555, 2020.

[65] Wang Y, Le H, Gotmare A D, et al. Codet5+: Open code large language models for code understanding and generation[J]. arXiv preprint arXiv:2305.07922, 2023.

[66] Croft R, Babar M A, Kholoosi M M. Data quality for software vulnerability datasets[C]//2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023: 121-133.

[67] Liu Z, Yang Z, Liao Q. Exploration On Prompting LLM With Code-Specific Information For Vulnerability Detection[C]//2024 IEEE International Conference on Software Services Engineering (SSE). IEEE, 2024: 273-281.

[68] He X, Tian Y, Sun Y, et al. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering[J]. Advances in Neural Information Processing Systems, 2024, 37: 132876-132907.

[69] Jiang Y, Zhang Y, Su X, et al. StagedVulBERT: Multi-Granular Vulnerability Detection with a Novel Pre-trained Code Model[J]. IEEE Transactions on Software Engineering, 2024.

[70] Gunasekar S, Zhang Y, Aneja J, et al. Textbooks are all you need[J]. arXiv preprint arXiv:2306.11644, 2023.

[71] Guo D, Ren S, Lu S, et al. Graphcodebert: Pre-training code representations with data flow[J]. arXiv preprint arXiv:2009.08366, 2020.

[72] Wang X, Wang Y, Mi F, et al. Syncobert: Syntax-guided multi-modal contrastive pre-training for code representation[J]. arXiv preprint arXiv:2108.04556, 2021.

[73] Roziere B, Gehring J, Gloeckle F, et al. Code llama: Open foundation models for code[J]. arXiv preprint arXiv:2308.12950, 2023.

[74] Fu M, Tantithamthavorn C. Linevul: A transformer-based line-level vulnerability prediction[C]// Proceedings of the 19th International Conference on Mining Software Repositories. 2022: 608-620.

[75] Nguyen V A, Nguyen D Q, Nguyen V, et al. Regvd: Revisiting graph neural networks for vulnerability detection[C]//Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings. 2022: 178-182.

[76] 邓枭，叶蔚，谢睿，等. 基于深度学习的源代码缺陷检测研究综述[J]. 软件学报, 2023, 34(2): 625-654.

[77] Brown T, Mann B, Ryder N, et al. Language models are few-shot learners[J]. Advances in neural information processing systems, 2020, 33: 1877-1901.

[78] Touvron H, Lavril T, Izacard G, et al. Llama: Open and efficient foundation language models[J]. arXiv preprint arXiv:2302.13971, 2023.

[79] Raffel C, Shazeer N, Roberts A, et al. Exploring the limits of transfer learning with a unified text-to-text transformer[J]. Journal of machine learning research, 2020, 21(140): 1-67.

[80] Nijkamp E, Pang B, Hayashi H, et al. Codegen: An open large language model for code with multi-turn program synthesis[J]. arXiv preprint arXiv:2203.13474, 2022.

[81] Li Y, Choi D, Chung J, et al. Competition-level code generation with alphacode[J]. Science, 2022, 378(6624): 1092-1097.

致谢

值此硕士论文完成之际，回首求学之路，感慨万千。这段宝贵的科研经历不仅培养了我的学术思维与问题解决能力，更让我学会了以冷静、理性的态度面对挑战。在此，我要向所有给予我帮助与支持的人表达最诚挚的谢意。

感谢我的导师高小鹏教授。您严谨的治学态度和创新视野为我的研究指明方向，在理论体系构建和研究方法应用上给予关键指导。其次，衷心感谢姜博教授。感谢您在我实验受阻时提供意见，在科研迷茫、焦虑时为我指引方向，以及从论文的选题、开题到定稿全程的悉心把关，使我得以顺利完成论文。您严谨的治学态度与平等开放的交流方式，始终是照亮我学术道路的明灯，必将对我未来的发展产生深远影响。同时感谢万寒老师和童超老师在论文修改过程中提出的宝贵建议，使本文得以不断完善。

感谢实验室同门和课程研讨中结识的良师益友，你们独到的见解和真诚的建议帮助我不断拓展思维边界。特别感谢室友曾丽芯、李文耕、陈新月同学三年来的陪伴与支持，让我的研究生生活充满温暖与欢笑。

感谢季浩宇同学始终如一的陪伴，我们相互启发与鼓励，共同见证着彼此的成长。感谢我的父母，始终尊重并支持我的每一个选择，你们毫无保留的爱永远是我最坚实的后盾。

最后，感谢那个秉持着「逢山开路，遇水架桥」信念，始终步履不停的自己。

学生生涯主线任务达成成就，新的世界副本《人生・社会篇》启动！